

Spiri



SpiriConfig

Release 0.1

Alex Davies

Nov 05, 2026

CONTENTS

1 What that buys you **3**

2 Getting started **5**

2.1 Installing SpiriConfig 5
2.2 Configuration 8
2.3 Advanced mode 12
2.4 The docker plugin 16
2.5 App store 20
2.6 Writing a plugin 25
2.7 Container plugins 28
2.8 Design notes 31
2.9 API reference 36
2.10 Revision history 86

Python Module Index **87**

Index **89**

Document control

Document number	**SPIRI-SPIRICONFIG-001**
Revision	**0.1**
Status	DRAFT – not for operational use
Effective date	2026-01-01
Issued by	Alex Davies

Uncontrolled when printed.

Plugin-based configuration and container management.

SpiriConfig gives you a web UI and a CLI over the services running on a machine. It is built around one rule:

> **Anything SpiriConfig can do, you must be able to do without it.**

This is not a slogan, it is a design constraint that shows up in the code. When you press **Up** on a stack in the web UI, SpiriConfig runs:

```
$ cd /srv/compose/whoami && docker compose -p whoami -f compose.yaml up -d
```

…and shows you that line while it runs, with a button to copy it. There is no hidden state, no database, and no bespoke on-disk format. If SpiriConfig were uninstalled tomorrow, everything it manages would keep working, and you would already know the commands to manage it.

That property is called *progressive enhancement*: the tool is a convenience on top of a system that is fully usable without it, never a dependency the system grows into.

CHAPTER

ONE

WHAT THAT BUYS YOU

You can manage a stack from SpiriConfig, from a plain shell, or from both on the same afternoon, and nothing gets confused. The compose project name we pass is the directory name, so the containers SpiriConfig starts are the *same* containers you get from running compose yourself in that directory:

```
$ spiriconfig docker up whoami      # these two do
$ cd /srv/compose/whoami            # exactly the same
$ docker compose up -d                # thing
```

Not sure what a button will do? Ask, without running it:

```
$ spiriconfig docker up whoami --show
cd /srv/compose/whoami && docker compose -p whoami -f compose.yaml up -d
```

CHAPTER

TWO

GETTING STARTED

```
$ uv sync
$ export SPIRICONFIG_DOCKER_COMPOSE_DIR=/srv/compose
$ spiriconfig serve
```

The web UI is at http://localhost:8080. To add a service, make a directory with a compose file in it – no CLI required, that is the point:

```
$ mkdir -p /srv/compose/whoami
$ $EDITOR /srv/compose/whoami/compose.yaml
```

It shows up in the UI on the next refresh.

2.1 Installing SpiriConfig

Every other service SpiriConfig manages lives in a container. SpiriConfig itself cannot: the thing that starts your containers on boot has to live on the host, start on boot itself, and update in place. So it installs itself as a systemd service.

Like everything else here, the installer works by building the commands you could have run by hand – a `uv tool install`, a unit file, `systemctl enable` – and it will show you every one of them before it runs anything. If SpiriConfig were not here, you would set the machine up with exactly these steps; the installer just types them for you.

`spiriconfig install` uses uv to place the program, so uv needs to be on the machine first.

2.1.1 Install it

From a machine that already has `spiriconfig` on its path (a checkout, or a throwaway `uvx spiriconfig`), run:

```
$ spiriconfig install
```

That installs the latest release from PyPI, writes a systemd unit and an environment file, and enables and starts the service. When it finishes, the web UI is running and will come back on every boot.

To see precisely what it will do without touching anything, add `--show`. It prints the `uv tool install`, the full unit file, the environment file with its path, and the `systemctl` commands – everything, in order, ready

to copy:

```
$ spiriconfig install --show
```

2.1.2 Two scopes: root or a single user

The installer looks at who you are and picks one of two installs. You do not choose; the choice *is* the security model (see *Authentication*).

- **As root**, it installs a system service under `/etc/systemd/system` that runs as root. Root can verify any account's password, so this is the only install where the PAM login is genuinely multi-user and the *users plugin* can manage accounts.
- **As a normal user**, it installs a `systemctl --user` service under your `~/.config` that runs as you. PAM can then only authenticate your one account – a single-operator box. It also runs `loginctl enable-linger` for you, without which a user service stops the moment you log out, which on a headless drone means it never really runs.

Either way, the login gate defaults to PAM. A single-operator box still gets a password in front of it; it simply only ever accepts that one account.

2.1.3 What it writes

Two files, both plain text you can read and edit:

	System (root)	User
Unit	`/etc/systemd/system/spiriconfig.service`	`~/.config/systemd/user/spiriconfig.service`
Environment	`/etc/spiriconfig/config.env`	`~/.config/spiriconfig/config.env`

The environment file is one `SPIRICONFIG_*` variable per line – the same settings documented under *Configuration*. To change how the installed service runs, edit that file and restart:

```
$ systemctl restart spiriconfig          # or: systemctl --user restart␣
↪spiriconfig
```

2.1.4 Options

The install questions all have flags, with sensible defaults:

Option	Default	Meaning
`--host`	`127.0.0.1`	Address to bind. Loopback by default; a real address needs a login.
`--port`	`8080`	Port to bind.
`--auth`	`pam`	Login gate: `pam` or `none`.
`--auth-grou`	`wheel`	Group whose members may log in, when run as root.
`--compose-d`	`/srv/compose` (root), `~/spiri-apps` (user)	Where the docker plugin looks for apps.
`--editable`, `-e`	off	Install the source in editable mode (for developing the installer).

The first positional argument is what to install, and it is anything uv accepts:

```
$ spiriconfig install                                    # latest release
$ spiriconfig install 'spiriconfig==0.1.0'               # a pinned version
$ spiriconfig install 'git+https://git.spirirobotics.com/Spiri/SpiriConfig@main
↪'
$ spiriconfig install -e .                               # a working checkout
```

Note

The installer refuses one combination outright: `--auth none` on a non-loopback address. That would leave the machine reachable off-box with no login and full control, and an install is durable – set up that way, it stays that way. Bind to loopback, or keep the login (the default). This is the same danger `spiriconfig serve` only *warns* about, escalated to a refusal because a service is permanent.

2.1.5 Updating

```
$ spiriconfig update
```

This upgrades the program via `uv tool upgrade` and restarts the service. The restart is handed to systemd to perform a moment later rather than run inline, because the process being restarted is the very one running the command – an inline restart would cut `update` off before it could tell you how it went.

`uv tool upgrade` re-resolves versions, which is what you want for a release install. A git-branch install is a different case: a moved branch is the same "version" to uv, so pass `--reinstall` to force a refetch of the new commit:

```
$ spiriconfig update --reinstall
```

`--show` works here too, printing the upgrade and restart commands without running them.

2.1.6 Managing the service afterward

Nothing about the installed service is special – it is an ordinary systemd unit. Use `systemctl` (add `--user` for a user install) as you would for anything else:

```
$ systemctl status spiriconfig
$ systemctl restart spiriconfig
$ systemctl disable --now spiriconfig    # stop it and remove it from boot
```

To remove SpiriConfig entirely, disable the service, delete the unit and environment files listed above, and run `uv tool uninstall spiriconfig`.

2.2 Configuration

SpiriConfig is configured entirely through environment variables. There is no config file to learn and none for us to rewrite behind your back, which means a systemd unit, a shell, a `.env` file, or a container runtime can all configure it the same way.

If a `.env` file exists in the working directory it is read, but real environment variables always win. Flags to `spiriconfig serve` win over both:

```
$ spiriconfig serve                          #
↪$SPIRICONFIG_HOST:$SPIRICONFIG_PORT
$ spiriconfig serve 127.0.0.1:8338           # HOST:PORT
$ spiriconfig serve 0.0.0.0                  # HOST, configured port
$ spiriconfig serve :8338                    # PORT, configured host␣
↪([::1]:8338 for IPv6)
$ spiriconfig serve --no-login-required      # SPIRICONFIG_AUTH=none
```

2.2.1 Core

Variable	Default	Meaning
SPIRICONFIG_HOST	127.0.0.1	Address the web UI binds to. Loopback by default; set 0.0.0.0 to expose it on the network.
SPIRICONFIG_PORT	8337	Port the web UI binds to.
SPIRICONFIG_LOG_LEVEL	INFO	DEBUG, INFO, WARNING, ERROR.
SPIRICONFIG_LOG_FILE	*(none)*	Also log to this file, rotated at 10 MB.
SPIRICONFIG_ADVANCED	false	Default for *advanced mode*, for someone who has not chosen.
SPIRICONFIG_STORAGE_SECRE	*(generated)*	Signs the cookie per-person settings are keyed on. Set it, or those settings reset on every restart.
SPIRICONFIG_AUTH	pam	none or pam. pam puts a login in front of every page. See *Authentication*.
SPIRICONFIG_AUTH_SERVICE	login	PAM service (a file under /etc/pam.d/) to authenticate against.
SPIRICONFIG_AUTH_GROUP	wheel	Group whose members may log in, *when SpiriConfig runs as root*. sudo on Debian.
SPIRICONFIG_TLS	auto	auto or off. auto serves a self-signed cert when exposed off loopback; off stays plain HTTP. See *Transport (TLS)*.
SPIRICONFIG_TLS_CERT	*(none)*	Path to a TLS cert to serve instead of a generated one. Set with SPIRICONFIG_TLS_KEY.
SPIRICONFIG_TLS_KEY	*(none)*	Path to the private key for SPIRICONFIG_TLS_CERT.

2.2.2 Authentication

By default every page requires a login, checked against the host's PAM stack — the same accounts that can ssh in or sudo, no separate user list of ours. Set SPIRICONFIG_AUTH=none to drop that gate, which only makes sense on loopback in a checkout with no other users on the box; spiriconfig serve warns if you bind a non-loopback address and turn it off.

Who can log in depends on whether SpiriConfig runs as root, because only root can verify another account's password:

- **As root**, it can authenticate any system user, so a group gates who counts as an administrator. Membership of SPIRICONFIG_AUTH_GROUP (default wheel) is the gate — set it to sudo on Debian, or to whatever group your admins are in.
- **Not as root**, it can only authenticate the one account it runs as. That is the only login the page will accept (it prefills the name for you), and the group setting does not apply.

login is used as the PAM service because it exists on essentially every system. A deployment that wants its own policy can drop a /etc/pam.d/spiriconfig file and set SPIRICONFIG_AUTH_SERVICE=spiriconfig.

Set `SPIRICONFIG_STORAGE_SECRET` when auth is on: it signs the session cookie, and without a stable one everybody is logged out every time the process restarts.

Note

This is authentication, not authorization. It only decides who may log in. Once logged in, everyone drives the same process with the same access — anyone you let in can do anything the UI can. Per-user permissions are not part of the model. See *design*.

2.2.3 Transport (TLS)

With `auth=pam` the login sends a real host password, so it must not travel over plain HTTP. These are appliances with no reverse proxy in front, so the TLS lives here: bind a non-loopback address and, by default (`SPIRICONFIG_TLS=auto`), SpiriConfig generates a **self-signed** cert and serves HTTPS. The cert is written once to the service's state directory (`/var/lib/spiriconfig/tls` as a system service, `~/.local/state/spiriconfig/tls` as a user one) and reused across restarts, so the browser warning is a one-time approval, not a per-boot one. A loopback checkout stays on plain HTTP, where a cert warning would be pure friction.

Be clear about what self-signed buys. It encrypts the wire, so a **passive** eavesdropper on the LAN cannot read the password or the session cookie. It does **not** stop an **active** man-in-the-middle: the browser warns, the operator clicks through, and an attacker presenting *their own* self-signed cert is clicked through just the same. Self-signed is encryption, not identity.

For a deployment that must resist an active attacker, give SpiriConfig a cert the browser can *validate* — a per-device cert from a CA your fleet trusts — with `SPIRICONFIG_TLS_CERT` and `SPIRICONFIG_TLS_KEY`. That is the only path where a substitute cert is rejected instead of clicked through, and the only one where `Strict-Transport-Security` is sent (HSTS on a self-signed origin would forbid the very click-through the operator needs and lock them out).

Set `SPIRICONFIG_TLS=off` only when something in front already terminates TLS — a reverse proxy — so SpiriConfig should not encrypt a second time. On a non-loopback bind with `pam` and `tls=off`, `spiriconfig serve` warns that passwords are going out in the clear.

2.2.4 Docker plugin

Every plugin namespaces its settings under its own prefix, so plugin config never collides.

Variable	Default	Meaning
`SPIRICONFIG_DOCKER_COMPOSE_`	`/srv/compose` as root, `~/spiri-apps` otherwise	Directory holding one subdirectory per compose project. Installed apps are symlinks in here.
`SPIRICONFIG_DOCKER_DOCKER_B`	`docker`	The docker executable. Set to `podman` to use podman.
`SPIRICONFIG_DOCKER_COMMAND_`	`300`	Seconds before a captured command is considered hung.

2.2.5 App store plugin

Variable	Default	Meaning
SPIRICONFIG_APPSTORE_STO	[]	JSON list of git URLs, or local paths, of *app stores* to offer before any is added.
SPIRICONFIG_APPSTORE_STO	/var/lib/spiriconfig/stores as root, ~/.local/share/spiriconfig/stores otherwise	Where store clones live. Not a cache: your edits to installed apps are commits in here.
SPIRICONFIG_APPSTORE_GIT_	git	The git executable.
SPIRICONFIG_APPSTORE_COM	300	Seconds before a git command is considered hung.

2.2.6 Where things live by default

The path defaults depend only on who runs SpiriConfig, and are the same ones `spiriconfig install` writes into the service's environment file – so `uvx spiriconfig serve` and the installed service, run as the same user, see the same apps:

	root	anyone else
Installed apps (SPIRICONFIG_DOCKER_COMPOSE	/srv/compose	~/spiri-apps
Store clones (SPIRICONFIG_APPSTORE_STORE	/var/lib/spiriconfig/stores	$XDG_DATA_HOME/spiriconfig/stores (~/.local/share/...)

The compose directory is created by the first app install if it does not exist.

A checkout overrides them. Running out of a checkout should not start managing the containers on the developer's actual machine, so `./scripts/test-data.sh` builds a disposable `test_data/` tree with an example app store in it, and appends lines to the checkout's (gitignored) `.env` pointing all three app settings at it.

2.2.7 What gets logged

Commands that *change* something – `up`, `down`, `restart`, `pull` – are logged at INFO, as the exact shell line that ran:

```
13:34:30 INFO     docker   $ cd /srv/compose/hello && docker compose -p hello -
↪f compose.yaml up -d
```

Read-only queries (statuses, listings) are logged at DEBUG, so the INFO log stays a clean record of what SpiriConfig actually did to the machine. Set `SPIRICONFIG_LOG_LEVEL=DEBUG` to see everything.

2.2.8 Running as a service

`spiriconfig install` writes a unit like the one below and enables it for you – see *Installing SpiriConfig*. This section shows the unit itself, because nothing about SpiriConfig is special here: it is a normal program that reads its environment, so you can also just write the unit by hand.

```
[Unit]
Description=SpiriConfig
After=docker.service
Wants=docker.service

[Service]
Environment=SPIRICONFIG_DOCKER_COMPOSE_DIR=/srv/compose
Environment=SPIRICONFIG_APPSTORE_STORE_DIR=/var/lib/spiriconfig/stores
Environment=SPIRICONFIG_APPSTORE_STORES=["https://github.com/spiri/spiri-apps"]
Environment=SPIRICONFIG_PORT=8080
ExecStart=/usr/local/bin/spiriconfig serve
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Note

SpiriConfig runs `docker` as whatever user it runs as. That user needs access to the docker socket. Granting docker socket access is equivalent to granting root, so run SpiriConfig somewhere you would be comfortable running `docker` by hand.

2.3 Advanced mode

One machine has to serve two audiences: someone who wants to restart a service without thinking about it, and someone who wants the exact `docker compose` line so they can go and do something cleverer with it. Advanced mode is the switch between those two views.

It is a toggle at the bottom of the sidebar. Off by default; the choice sticks.

2.3.1 What it is not

⚠ DANGER

Advanced mode is not a permission system. It is not a security boundary. It must never be used as either.

The toggle is self-service. It sits in the sidebar of every page, and anybody looking at the web UI can turn it on. One click, and every advanced feature is right there. Nothing is withheld from anyone.

So a person is never *prevented* from editing a compose file by having advanced mode off. They flip the

switch and edit it in the browser, like anyone else. No shell, no CLI, no special knowledge – the switch is labelled and visible.

And underneath even that: hiding an element is a *presentation* fact. The page talks to the server over a websocket, and what a browser chooses to display has no bearing on what a client can send down it. Anyone who wants the capability has it, whatever the page is currently rendering.

Advanced mode gives people who want a simpler page a simpler page. That is the entire feature. To actually stop someone doing something, see *below* – it looks nothing like this.

Everything stays reachable from the CLI too, of course – that is what makes *progressive enhancement* true. But the CLI is not the loophole here, and neither is the browser. Treat advanced mode as a decluttering preference, on the same footing as a dark-mode switch.

2.3.2 If you need real permissions

Not this. Permissions are not planned. If they ever landed, they would belong to the OS, not to us – file modes, group membership, whether someone is in the `docker` group at all – with no application-level role model, no ACL table, and nothing of ours to get wrong. But that is a hypothetical, not a roadmap: today PAM only decides who may log in, and everyone who does is equal.

That is the same principle as the rest of SpiriConfig. We drive the machine by running the commands a human would run, so a person should be able to do exactly what their shell account could do, and nothing more. An app-level permission model would be a second, weaker source of truth sitting in front of a capability the kernel already governs – and the first time the two disagreed, the kernel would win and our UI would be lying.

So: capability is decided by *who you are on the machine*. Advanced mode only decides *how much the page shows you*. Keeping those apart is what stops the UI from merely looking safe.

2.3.3 What is behind it

Feature	Why
Editing a compose file	The most dangerous thing on the page. Still available from a shell: `$EDITOR "$(spiriconfig docker config whoami)"`.
Editing an app's `.env`	The settings form is the app author's list of knobs, which is a good default and a poor cage. The panel under the form shows the exact bytes about to be written, and lets you type in them – variables the author never declared, comments, anything. Still available from a shell: `$EDITOR "$(spiriconfig docker env whoami)"`.
The raw command display	The `cd … && docker compose …` line and its copy button, in every action dialog. Developers want it; for everyone else it is noise.
Settings fields marked `advanced:`	The app author's own judgement about which of *their* knobs a developer wants: log levels, debug flags, tuning. A form asking eleven questions when three of them are the ones anybody came for is a form people learn to click past. See below.

Note what is *not* hidden: logs, and the settings form itself. A regular user is exactly the person who needs to see why a service failed, and the output of a command is not the same thing as the invocation that produced it. And an app author decided which knobs are safe to turn, so turning one is an ordinary act – what is advanced is the file they land in.

2.3.4 An app can mark its own settings advanced

The only entry in that table an app author writes themselves. A field in `x-spiri-settings` with `advanced: true` on it is rendered only in advanced mode:

```
x-spiri-settings:
  - env: GRAFANA_PORT          # everyone is asked this
    widget: number
    label: HTTP port

  - env: GRAFANA_LOG_LEVEL     # only a developer is
    widget: select
    label: Log level
    options: [debug, info, warn, error]
    default: info
    advanced: true
```

It hides the *widget*, and nothing else. The variable is still read from the `.env`, still written back when the form is saved – a value you cannot currently see is not a value we will quietly drop – and the CLI lists and sets it whatever the switch says:

```
$ spiriconfig docker settings grafana
GRAFANA_PORT       3000   (default)
GRAFANA_LOG_LEVEL  info   (default, advanced)
```

Which is the same bargain as the rest of this page. `advanced:` is an app author tidying a form, not one deciding who is allowed to configure their app; if it ever became the latter, it would be a permission system built out of somebody's opinion about clutter.

2.3.5 Setting the default

`SPIRICONFIG_ADVANCED` sets the starting position for someone who has not chosen yet – so a developer image can ship with it on, and a customer image with it off, from the same code:

```
$ SPIRICONFIG_ADVANCED=true spiriconfig serve
```

A person's own choice always beats this default.

To make that choice survive a restart, set a stable secret – the cookie identifying a browser is signed with it:

```
$ export SPIRICONFIG_STORAGE_SECRET="$(openssl rand -base64 32)"
```

Without one, SpiriConfig generates a temporary secret, warns, and everybody's setting resets when the process restarts.

2.3.6 Using it in a plugin

```
from spiriconfig import advanced

ui.button("Up", on_click=...)          # everyone

with advanced.only():
    ui.button("Edit", on_click=...)    # developers only
```

Elements inside *only()* are *bound* to the setting rather than conditionally created, so the toggle takes effect instantly, without rebuilding the page or losing whatever the person was in the middle of.

For a single element, *mark()* does the same thing and chains:

```
advanced.mark(ui.label(str(command))).classes("font-mono")
```

And to branch on it, *enabled()*:

```
if advanced.enabled():
    ...
```

Prefer `only()` and `mark()` where you can: because they bind rather than branch, they respond to the toggle without a re-render, and `enabled()` does not.

2.3.7 When users arrive

Today SpiriConfig has no users, so "this person" means "this browser", identified by a session cookie. That is the finest-grained identity available, and it is the limitation user support will lift.

The code is built for that day. Nothing reads browser storage directly; every preference goes through one seam, *spiriconfig.preferences*. Adding user support means registering a different store, once:

```
from spiriconfig import preferences

class DatabasePreferences:
    def __init__(self, user_id: str) -> None:
        self.user_id = user_id

    def get(self, key, default):
        return db.fetch_preference(self.user_id, key, default)

    def set(self, key, value) -> None:
        db.store_preference(self.user_id, key, value)

preferences.use(lambda: DatabasePreferences(current_user().id))
```

The factory is called per access, not once, so it can resolve *whoever is being served right now* – which is exactly what a per-user setting needs.

No plugin changes. `advanced.only()` keeps working, and the setting silently becomes a property of the

person rather than of the browser they happen to be sitting at. There is a test (`test_advanced.py`) that registers a store keyed on something other than a browser and asserts advanced mode follows it, precisely so this promise does not quietly rot before anyone tries to use it.

2.4 The docker plugin

Manages the docker compose projects in one directory.

2.4.1 The model

A **stack** is a subdirectory of `SPIRICONFIG_DOCKER_COMPOSE_DIR` that contains a compose file. That is the whole model.

```
/srv/compose/
├── whoami/
│   └── compose.yaml
├── nextcloud/
│   ├── compose.yaml
│   └── .env
└── notes.txt          <- ignored: not a directory
```

Any of `compose.yaml`, `compose.yml`, `docker-compose.yaml`, or `docker-compose.yml` works, with the same precedence docker compose itself uses. A directory with no compose file is ignored, not an error.

There is no "install", "register", or "enable" step, because there is no state for us to keep in sync with the disk. Adding a service is creating a directory; removing one is deleting it. `mkdir` is a supported workflow.

2.4.2 Running and not running

There is no concept of "enabled" beyond what docker already tracks. A stack is either up or it is not, and `docker compose` is the authority on which:

- **running** – every container is up
- **partial** – neither all up nor all dead: one crashed, one is restarting, or the stack is still coming up
- **stopped** – containers exist, and all of them are dead
- **down** – no containers exist yet

Note

`partial` covers "in flux", not just "broken". `docker compose up -d` returns as soon as it has *started* the containers, which is a moment before the daemon reports them as running – so a status check immediately after an up can honestly see `partial`. Ask again shortly.

2.4.3 Commands

Each of these maps to one `docker compose` invocation. Add `--show` to any of them to print that invocation instead of running it.

```
$ spiriconfig docker list                  # every project, and its status
$ spiriconfig docker up whoami           # docker compose up -d
$ spiriconfig docker down whoami         # docker compose down
$ spiriconfig docker restart whoami      # docker compose restart
$ spiriconfig docker pull whoami         # docker compose pull
$ spiriconfig docker logs whoami -f      # docker compose logs --tail=200 --
↪follow
$ spiriconfig docker ps whoami           # docker compose ps
$ spiriconfig docker config whoami       # print the path to the compose file
$ spiriconfig docker exec whoami whoami    # docker compose exec whoami /bin/sh
$ spiriconfig docker attach whoami whoami # docker compose attach whoami
```

`config` prints a path rather than opening an editor, so that your own tools can do the part they are better at:

```
$ $EDITOR "$(spiriconfig docker config whoami)"
```

2.4.4 Getting inside a container

Two buttons, both *advanced*-only, both opening a real terminal in the browser. They are not the same thing, and the difference is worth knowing before you press one.

Exec starts a *new* process next to the app — a shell, by default. You can exit it and nothing has happened to the container. It defaults to `/bin/sh` rather than `bash` because `sh` is the shell a container actually has: Alpine images ship busybox and no bash at all. The command is a text box, not a shell button, because `docker compose exec` runs anything the image has:

```
$ spiriconfig docker exec grafana grafana                  # a shell
$ spiriconfig docker exec grafana grafana -- ls -la /etc   # or anything else
```

Put `--` before a command with options of its own, or they will be read as ours.

Attach connects you to the process the container already *is* — pid 1, the app itself. It is how you reach a REPL an app serves on its stdin, and how you see output that never goes near the logs. Your keystrokes go to the app, so what Ctrl-C does is between you and that process. We pass no `--sig-proxy` and no `--detach-keys`: it is the plain command, and it behaves exactly as it does when you type it.

Both only offer services that are **running**, since both need a live process.

The orphan, and what we do about it

Note

`docker compose exec` does not clean up after itself, and it never has.

When the client that started an exec goes away, the process it started inside the container keeps running. Forever — there is no timeout, and nothing reaps it short of the container stopping. The exec'd process is not the client's child; it belongs to the container, and the daemon does not tear an exec down when the API connection drops. The docker API can *start* an exec and *inspect* an exec, and offers no way at all to *kill* one.

This is moby/moby#9098, open since **2014**, along with #29700 (the orphan, reparented to PPID 0) and #35703 (the request for a `docker exec kill` that would make all of this unnecessary).

Nobody meets it at a terminal, because at a terminal you type `exit`, and a shell that exits is clean. **A browser tab has no `exit`** — closing it *is* the hangup. So the rare accident becomes the ordinary path, and without help every visit to the Exec button would leave a shell running inside your container.

So the web session runs the exec wrapped in a tiny supervisor that records its pid inside the container, and kills it when you close the dialog or the tab. The command line the page *shows* you is the plain one — the one the CLI runs, and the one worth copying — because that line should be about your container, not about our workaround.

It is worth knowing that the CLI does **not** do this. `spiriconfig docker exec` is exactly `docker compose exec`, orphan and all: if you kill your terminal instead of typing `exit`, you will leave a shell behind, precisely as you would running docker by hand.

2.4.5 Editing a compose file

The web UI can edit a stack's compose file. Two things about how it saves:

Your file is text, and stays text. We never parse the YAML and write it back out, so your comments, ordering, and formatting survive a save untouched.

A file that compose would reject is never written. On save we check that it parses as YAML, then ask `docker compose config` whether it is actually valid. If either check fails, the original file is restored and you get the error. A bad save cannot leave you with a stack that will not start.

This is the one place SpiriConfig does something a plain editor does not, and it is still not a lock-in: the file on disk is an ordinary compose file, and `vim` remains a completely valid way to edit it.

2.4.6 Settings forms (`x-spiri-settings`)

An app can declare a settings form for its own `.env` file, either inline in the compose file or in a sidecar next to it – see *`declared()`* for exactly how the two are found and why declaring both is an error. Each item is one field:

```
x-spiri-settings:
  - env: GRAFANA_PORT          # the .env variable this field reads and writes
    widget: number              # see the table below
    label: HTTP port            # optional; defaults to a readable form of␣
```

(continues on next page)

(continued from previous page)

```
↪`env`
    help: Shown under the widget.
    default: "3000"
    min: 1024
    max: 65535
    advanced: false              # see advanced.md
```

`env` is the only required key. Everything else, including `widget`, has a default – see *`Field`* for the full set (`options`, `min`, `max`, `step`, `required`, `pattern`, `advanced`).

Widgets

`widget:` is a direct name, not a type the form guesses a control from – see the module docstring in `spiriconfig_docker/widgets.py` for why. The table below is generated at build time straight from *`spiriconfig_docker.widgets.REGISTRY`* – one row per widget, each one's "Renders" cell pulled from that widget's own builder docstring – by the `widget-table` directive in `docs/_ext/widget_table.py`. A widget added to the registry with no docstring fails the docs build rather than landing on the page as a blank row, and one added and never wired into this table is not possible: there is no second list to fall out of sync with.

widget:	Renders	Needs options:
`input`	A single-line text box.	
`password`	A text box with the value masked, and a toggle to reveal it.	
`textarea`	A multi-line text box.	
`number`	A number box. Respects `min`, `max`, `step`.	
`slider`	A slider, plus the label and the readout it does not come with.	
`switch`	An on/off switch.	
`checkbox`	A checkbox.	
`select`	A dropdown, fixed to `options`.	Yes
`radio`	Radio buttons, fixed to `options`.	Yes
`toggle`	A button group, fixed to `options`.	Yes
`color`	A color picker.	
`video_device`	A dropdown of video sources: cameras found on this machine, URLs an app	Optional

`select`, `radio`, and `toggle` are validated against their `options:` on save – a value that is not one of the choices is rejected. `video_device` is not: a camera present when the form was saved may be unplugged by the time the `.env` is read again, and a typed-in URL was never on any fixed list to begin with, so it is not treated as an error.

An `advanced: true` field is hidden unless *advanced mode* is on – hidden, not removed: it is still read from the `.env`, still written back on save, and still shows up in `spiriconfig docker settings <name>` regardless of the switch.

2.4.7 Using podman

The plugin shells out to whatever `SPIRICONFIG_DOCKER_DOCKER_BIN` names, and only uses `compose` subcommands:

```
$ export SPIRICONFIG_DOCKER_DOCKER_BIN=podman
```

2.5 App store

An app store is a git repository with one directory per app, each containing a compose file. That is the entire format. There is no manifest, no index, and nothing to register: if you can push a directory with a `compose.yaml` in it, you have published an app.

```
spiri-apps/
├── whoami/
│   └── compose.yaml
├── nextcloud/
│   ├── compose.yaml
│   └── nginx.conf
└── traefik/
    └── compose.yaml
```

Which means you can use one without SpiriConfig at all:

```
$ git clone https://github.com/spiri/spiri-apps
$ cp -r spiri-apps/whoami /srv/compose/
$ cd /srv/compose/whoami && docker compose up -d
```

Everything below is a convenience on top of that.

2.5.1 Pointing at a store

```
$ export SPIRICONFIG_APPSTORE_STORES='["https://github.com/spiri/spiri-apps"]'
$ spiriconfig appstore check
```

`check` clones stores you have not got yet, and fetches the ones you have. It is safe to run at any time: fetching only updates git's idea of what the remote has, and changes no installed app – it is what makes `list`'s "update available" markers truthful. The web UI fetches every store once on startup, and again whenever you press **Check for updates**, so the same markers stay honest there without you running anything.

You can configure several stores. If two of them have an app with the same name, say which you mean: `spiriconfig appstore install spiri-apps/whoami`.

2.5.2 Installing

```
$ spiriconfig appstore list
spiri-apps
  nextcloud  2026-03-14
```

(continues on next page)

(continued from previous page)

```
  traefik    2026-01-08
  whoami     1.10.1

$ spiriconfig appstore install whoami
Installed spiri-apps/whoami as whoami.
Start it with: spiriconfig docker up whoami
```

Installing makes a symlink, and nothing else:

```
$ spiriconfig appstore install whoami --show
ln -s /var/lib/spiriconfig/stores/spiri-apps/whoami /srv/compose/whoami
```

The app is now an ordinary stack. The docker plugin has no idea it came from a store, so everything works on it exactly as it would on a directory you made yourself:

```
$ spiriconfig docker up whoami
$ cd /srv/compose/whoami && docker compose up -d   # the same containers
```

Installing does not start anything. Two apps that both want port 8080 will not fight until you tell them to, and `--as` gives one of them another name:

```
$ spiriconfig appstore install whoami --as whoami-test
```

2.5.3 Versions

An app's version is a **label**. Nothing depends on it, and you never have to answer the unanswerable question of what "version 2" of a set of containers means.

If the store's author set one, that is what you see:

```
x-spiri-config-version: "1.10.1"
services:
  whoami:
    image: traefik/whoami:v1.10.1
```

If they did not, you get the date of the last commit that touched the app, which is always available and always moves when the app does.

Updates are not decided by either of these. An app has an update when git says its files differ from the store's, which is exact.

2.5.4 Updating

The interesting case is the one where you have edited an app. You are *supposed* to edit apps – a store cannot know your ports, your paths, or your hardware.

Suppose you changed whoami's port, and the store bumped its image:

```
$ spiriconfig appstore diff whoami
--- Your changes ---
-      - "8080:80"
+      - "9080:80"

--- Store changes since your version ---
-    image: traefik/whoami:v1.10.1
+    image: traefik/whoami:v1.11.0
```

```
$ spiriconfig appstore update
```

You keep your port, and you get their image. That is a three-way merge, done by git, because an installed app lives in a git working tree and your edits to it are commits. Nothing clever is happening here, which is the point.

`update` rewrites files on disk. **It does not restart anything.** Your containers keep running the old definition until you say otherwise:

```
$ spiriconfig docker pull whoami && spiriconfig docker up whoami
```

This also means updates are per-store, not per-app: pulling the store updates every app's *files*, and you choose which apps to actually restart. That is the decision you cared about anyway.

When it conflicts

If you and the store changed the same lines, git cannot decide for you, and it does not pretend to:

```
$ spiriconfig appstore update
CONFLICT (content): Merge conflict in whoami/compose.yaml

Update stopped: your edits conflict with the store's.
  /var/lib/spiriconfig/stores/spiri-apps/whoami/compose.yaml
```

The file now has conflict markers in it:

```
services:
  whoami:
<<<<<<< HEAD
    image: traefik/whoami:v1.10.1-custom
=======
    image: traefik/whoami:v1.11.0
>>>>>>> @{upstream}
```

Nothing can start in this state, and that is a feature: conflict markers are not valid YAML, so `docker compose` refuses the file, and so does SpiriConfig's editor. You cannot accidentally run a half-merged app.

Edit the file, keep what you want, delete the `<<<<<<<`, `=======` and `>>>>>>>` lines, then:

```
$ spiriconfig appstore resolve
```

If any markers are left, this refuses and tells you which files. Running it too early cannot commit a broken app.

Or change your mind entirely:

```
$ spiriconfig appstore resolve --abort
```

That puts everything back exactly as it was before the update. **Your own edits survive**, because they were committed before the merge began.

If you would rather just take the store's version and throw your changes away:

```
$ spiriconfig appstore update --discard-local
```

2.5.5 Uninstalling

```
$ spiriconfig appstore uninstall whoami --show
rm /srv/compose/whoami
```

That removes a symlink. It deletes no app files, and no data. It also does not stop anything – run `spiriconfig docker down whoami` first, or its containers will outlive the thing that defined them.

SpiriConfig will refuse to uninstall a directory it did not create. If `/srv/compose/whoami` is a real directory, it is yours, and removing it is your call to make with `rm`.

2.5.6 Adopting

Adopting an app takes it out of the store and makes it yours.

```
$ spiriconfig appstore adopt whoami
Adopt whoami from spiri-apps?

    rm /srv/compose/whoami
    cp -r /var/lib/spiriconfig/stores/spiri-apps/whoami /srv/compose/whoami

whoami stops tracking the store for good: no more updates, and nothing here
will touch it again. There is no undo -- afterwards `spiriconfig appstore␣
↪uninstall`
will refuse it too, because the directory will be a real one we did not create.

Adopt it? [y/N]:
```

The symlink is replaced with a real copy, and the app becomes an ordinary compose project that you own completely. **It will never update again.** Nothing in the app store will touch it, and nothing in the app store will clean it up either – removing it afterwards is a `rm -rf` you do yourself.

It is the one irreversible thing here, which is why it is the one thing that asks. `--yes` skips the question, for scripts.

Use it when an app has diverged so far that merging is a chore, or when you want to take something the store is not going to follow.

2.5.7 A note for store authors

Prefer **named volumes** to relative bind mounts.

An installed app is a symlink into the store's git checkout, so a compose file that says `./config:/config` makes docker create `config/` *inside that checkout*. It works, and SpiriConfig ignores those directories when deciding whether you have edited an app – but it puts user data inside a git repository, which is not where anyone expects to find it.

```yaml
services:
  nextcloud:
    image: nextcloud:29
    volumes:
      - nextcloud-data:/var/www/html    # good
      # - ./data:/var/www/html          # works, but lands in the git checkout

volumes:
  nextcloud-data:
```

Set `x-spiri-config-version` if you have a version worth naming. If you do not, leave it out – the commit date is a perfectly good label, and a stale version number is worse than none.

2.5.8 Settings

Setting	Default	What it is
SPIRICON-FIG_APPSTORE_STORE	[]	JSON list of git URLs (or local paths) to offer before any is added.
SPIRICON-FIG_APPSTORE_STORE	/var/lib/spiriconfig/stores as root, ~/.local/share/spiriconfig/stores otherwise	Where clones live.
SPIRICON-FIG_APPSTORE_GIT_E	git	The git executable.
SPIRICON-FIG_APPSTORE_COMMA	300.0	Seconds before git is considered hung.

The store directory is not a cache you can blow away casually: your edits to installed apps live there, as commits. It is a git repository, and you can `cd` into it and use it as one.

A checkout of this repository points these at `test_data/` instead, through the `.env` that `scripts/test-data.sh` writes, so it does not reach for `/var/lib` and `/srv` and manage the containers on your actual machine – see *configuration*.

2.5.9 Trying it, from a checkout

```
$ ./scripts/test-data.sh
$ uv run spiriconfig appstore check
$ uv run spiriconfig appstore install whoami
$ uv run spiriconfig docker up whoami
$ curl localhost:8080
Hostname: 2d5bcd6f2629
IP: 172.18.0.2
GET / HTTP/1.1
```

`scripts/test-data.sh` copies `examples/store/` into `test_data/example-store` and `git init`s it, because a store has to be a git repo before it can be cloned, and points the checkout's `.env` at `test_data/`. The whole tree is gitignored and disposable: delete `test_data/` and run the script again.

The example store carries `whoami` (which answers with a description of your request – the fastest way to tell whether any of this works), `traefik`, and `nextcloud` (which has state, and shows what named volumes are for).

2.6 Writing a plugin

Everything a user would call a feature lives in a plugin. The core only discovers plugins, gives them somewhere to put a CLI and a page, and runs commands for them. The bundled docker plugin is loaded by exactly the machinery described here – there is no privileged built-in path, so if it works for docker it works for yours.

2.6.1 A plugin in full

```python
# src/spiriconfig_tailscale/__init__.py
import typer
from nicegui import ui

from spiriconfig.commands import Command, run
from spiriconfig.plugins import Plugin

cli_app = typer.Typer(help="Manage tailscale.")


def status_command() -> Command:
    """Build the command. Do not run it -- see the design notes."""
    return Command(argv=["tailscale", "status"])


@cli_app.command()
def status() -> None:
    """Show tailscale status."""
    typer.echo(run(status_command()).stdout)
```

(continues on next page)

(continued from previous page)

```python
class TailscalePlugin(Plugin):
    name = "tailscale"
    title = "Tailscale"
    description = "Show and manage the tailscale connection."
    icon = "vpn_lock"

    def cli(self) -> typer.Typer:
        return cli_app

    def page(self) -> None:
        ui.label("Tailscale").classes("text-2xl font-bold")
        ui.code(run(status_command()).stdout)
```

Register it:

```toml
[project.entry-points."spiriconfig.plugins"]
tailscale = "spiriconfig_tailscale:TailscalePlugin"
```

Install it, and it is there:

```
$ pip install -e .
$ spiriconfig plugins
docker     cli,web   Start, stop, and edit docker compose projects.
tailscale  cli,web   Show and manage the tailscale connection.

$ spiriconfig tailscale status
```

Installing the distribution is what makes a plugin available; uninstalling it is what removes it. There is no plugin registry to edit.

2.6.2 The interface

Subclass *`Plugin`* and set `name`, `title`, and `description`. Optionally set `icon` to a Material icon name, which is what the sidebar shows beside your title; it defaults to a generic plug. Then provide either or both of:

`cli()`
: A `typer.Typer` app, mounted at `spiriconfig <name> ...`.

`page()`
: Called inside a NiceGUI page route at `/<name>`. The shell gives you the whole main area – everything right of the sidebar is yours, heading included.

Both are optional. A plugin with only a `cli()` is fine and gets no sidebar entry (it still appears on the index page). A plugin with only a `page()` is *technically* fine and is almost always a mistake – see below.

2.6.3 The rules

1. Do the work by running the command a human would run.

Build a *Command* and hand it to *run()* or *stream()*. Do not reach for a Python API when a command line exists – not `docker-py`, not `requests` against a local socket. The point is not that subprocesses are elegant; it is that a command is something the user can read, copy, and run without us. A Python API call is not.

2. Build commands separately from running them.

Note that `status_command()` above returns a `Command` rather than running one. This is what lets the UI show the user what it is about to do, lets `--show` print it, and lets your tests assert on the exact command line without the underlying tool installed anywhere. The docker plugin tests do exactly this, and most of them need no docker daemon at all.

3. Never make the web UI the only way to do something.

If your page has a button, there must be a way to do that thing from a shell – ideally your `cli()`, but "run this documented command" counts too. A feature that only exists behind a mouse click is a feature the user cannot script, cannot automate, and cannot fix at 3am over a broken SSH connection.

4. Put developer-facing clutter behind advanced mode – and nothing else.

```
from spiriconfig import advanced

ui.button("Up", on_click=...)          # everyone
with advanced.only():
    ui.button("Edit", on_click=...)    # developers only
```

Advanced mode is a display filter, not a permission – the CLI still does everything, whatever the toggle says. If you are hiding a button because someone *should not be allowed* to press it, you want authorisation, and that is a unix account, not this.

5. Log through loguru, bound to your plugin.

```
from loguru import logger
log = logger.bind(plugin="tailscale")

run(status_command(), log=log)
```

Do not configure sinks; the core does that.

6. Take your settings from the environment.

Namespace them under your own prefix, so plugins cannot collide:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class TailscaleSettings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="SPIRICONFIG_TAILSCALE_")
    tailscale_bin: str = "tailscale"
```

2.6.4 Failure is contained

A plugin that fails to import, blows up when constructed, or is not actually a `Plugin` is logged and skipped. It does not take down the CLI or the web UI – the rest of the app loads, and you get a loud reason why yours is missing. A page that raises while rendering is caught and shown as an error on its own page.

You still have to write a working plugin. But a half-written one will not lock you out of the machine while you do.

2.7 Container plugins

A plugin does not have to be Python that shares our interpreter. It can be a container, in any language, that serves a web page – SpiriConfig finds it, proxies it into the shell, and frames it in an iframe next to every other plugin's page. There is nothing special about this compared to an *entry-point plugin*: it is a second, independent way to plug in, not a workaround.

The whole point is that the two do not share a dependency set. An entry-point plugin shares our interpreter, so it shares our resolved `nicegui`/`pydantic` pins, and we cannot bump either without a chance of breaking every third-party one. A container plugin ships its own image with its own runtime – its own language, even – and the only thing it shares with us is a wire contract.

2.7.1 A plugin is an app with labels on it

If a plugin is a container, it is a compose app with some labels, and the *app store* already knows how to install compose apps from a git repo. Nothing new is needed on that side – installing a container plugin *is* installing an app:

```yaml
services:
  ui:
    image: ghcr.io/someone/spiriconfig-netplan
    labels:
      spiriconfig.plugin.name: netplan
      spiriconfig.plugin.title: Network
      spiriconfig.plugin.icon: lan
      spiriconfig.plugin.port: "8080"
    volumes:
      - /etc/netplan:/etc/netplan   # trusted code; mounts what it needs
```

```
$ spiriconfig appstore install netplan
$ spiriconfig docker up netplan
```

That's it. There is no separate plugin registry, no enable flag, and no install step beyond what installing any app already does:

question	answered by	who does the work
what plugins exist?	`docker ps --filter label=spiriconfig.plugin.name`	docker
install one	install an app from a store	the app store, unchanged
uninstall	remove the symlink	the app store, unchanged
is it running?	is the container up?	docker
restart when it dies	`restart: unless-stopped` in its compose file	docker
where did it come from?	`readlink` + `git`	the app store, unchanged
update it	`git merge`	the app store, unchanged

2.7.2 Discovery

`spiriconfig.discovery` polls `docker ps` for containers carrying `spiriconfig.plugin.name`, then `docker inspect` to read the rest of the labels and the container's own IP on its docker network:

```
$ docker ps -q --filter label=spiriconfig.plugin.name
```

A container missing `spiriconfig.plugin.port`, or without a network IP yet, is logged and skipped – one misconfigured plugin does not blank the sidebar for the rest. This rescans on an interval (`SPIRICONFIG_PLUGIN_DISCOVERY_INTERVAL`, default 5 seconds; see *Settings* below), so an app installed or stopped while SpiriConfig is running appears or disappears in the sidebar on its own – no restart needed. A machine with no docker simply reports no container plugins; discovery logs and returns nothing rather than failing.

2.7.3 The contract

A plugin author owes three things:

1. Declare yourself with labels – `name`, `port` (required); `title`, `icon` (optional, default to the name and a generic web icon).

2. Work correctly behind a reverse proxy at a path prefix. SpiriConfig proxies the container at `/plugin/<name>/...`, on the same origin as the shell – not a port of its own. Same origin is why cookies, the PAM login gate, and (below) the address bar sync all just work; a different port would be a different origin and lose all three. The proxy sets `X-Forwarded-Prefix: /plugin/<name>` on every request, so a framework that honours that header emits correct URLs with no idea it is behind us. NiceGUI does this thoroughly, so a NiceGUI plugin needs no URL-handling code at all to port over.

3. Include one script tag, if you want deep links. An iframe is a separate browsing context with its own URL, so without help the shell's address bar never moves and the back button steps through the *iframe's* history instead of the shell's. `/plugin-sdk/shell.js` fixes both: it mirrors the plugin's own path up to the parent's address bar with `history.replaceState`, same-origin, no `postMessage` handshake needed.

```
<script src="/plugin-sdk/shell.js"></script>
```

Optional – a plugin that skips it still works, it just sits at a URL that never changes as you navigate inside it.

2.7.4 What SpiriConfig proxies

`spiriconfig.proxy` forwards two kinds of traffic between `/plugin/<name>/...` and the container's own root:

- **HTTP**, streamed both ways with `httpx` so a large download or an SSE stream is never buffered whole.
- **WebSocket**, relayed frame-for-frame – the part that makes a live UI (NiceGUI's socket.io, or anything else riding a websocket) work at all.

The shell page a user actually navigates to is `/app/<name>/...` – distinct from `/plugin/<name>/...`, which is the iframe's `src` and returns the container's raw bytes. `/app/<name>` renders the sidebar and frames the proxied content in an iframe that fills the rest of the page; a name with no live target (container stopped, or a stale link) gets an honest "not available" card instead of a blank frame.

2.7.5 This is not a sandbox

A plugin is trusted code the operator chose to install, same as an entry-point one – SpiriConfig is not defending against a hostile plugin. The proxy forwards the plugin's cookies and inherits whatever auth sits in front of the shell; it is plumbing, not a security boundary. A container plugin bind-mounting `/var/run/docker.sock` or `/etc` is not a hole in the design, it is the design: the same freedom an entry-point plugin already has by sharing our process.

2.7.6 The CLI face

A container plugin cannot add a Typer subcommand, and does not need to – its entrypoint is its CLI:

```
$ docker exec spiriconfig-netplan netplan-cli show
```

A command a human could have run, which is the same test every other plugin is held to (see *Writing a plugin*).

2.7.7 Settings

Setting	Default	What it is
`SPIRICONFIG_PLUGIN_DISCOVERY_INTERV`	`5.0`	Seconds between rescans for container plugins. `0` scans once at startup and never again.

2.7.8 Still open

The labels, discovery, and proxy described above are implemented and in use. A few things around the edges are not yet decided – an asset-cache cost per plugin prefix, a dedicated "container is down" card, and whether the label contract itself should carry a version (`spiriconfig.plugin.api`) so it can change without breaking

every plugin at once. See `NOTES-out-of-process-plugins.md` for the reasoning still in flux; it is scratch, not documentation, and should not be relied on for anything this page already states as settled.

2.8 Design notes

Why the code looks the way it does. Read this before changing the core.

2.8.1 Progressive enhancement is the whole point

Most management UIs are a trap. You click a button, something happens, and now the state of your machine lives in the tool's database. To understand your own system you have to understand the tool. To leave, you have to reverse-engineer what it did. The tool has stopped being a convenience and started being a dependency.

SpiriConfig is built so that cannot happen. The rule:

> Anything SpiriConfig can do, the user must be able to do without it.

Every design decision below follows from that one, and any change that erodes it is a bug, however convenient it seems.

2.8.2 Consequences

Traditional devops workflows continue to work

You can manage a drone with ansible, terraform, whatever sysadmin tools make sense for managing your cluster of drones and sensor platforms.

We shell out, on purpose

Everything SpiriConfig does to the system is done by running the command a human would have run. We do not use `docker-py`; we run `docker compose`.

This looks primitive, and it is deliberate. A subprocess call is something we can *show the user*, and they can copy it, run it, and understand it. A library call is invisible and unreproducible. `str(Command)` renders a copy-pasteable shell line – properly quoted, `cd` included – and there is a test that pipes it through a real shell to prove it is not decorative.

Output goes to a terminal, because the command expects one

The UI streams command output into xterm.js, and the command runs attached to a **pseudo-terminal** rather than a pipe. Both halves of that are necessary, and the first one is the surprising one.

Docker asks whether its output is going to a terminal, and *changes what it says* based on the answer. On a pipe it emits a flat transcript. On a tty it draws progress bars, redraws layer-download status in place, and colours each service differently. So piping the output does not give you a plain version of the same information — it gives you a program that concluded you were not worth talking to properly. `docker compose pull` is the clearest case: through a pipe it is a wall of "Pulling"; on a tty it is a live picture of what is happening.

Since the entire project is built on *running the command a human would have run*, running it in a way that makes it behave differently than it would for that human is a bug in the premise. *`stream_pty()`* gives it a terminal.

That decision forces the second: the bytes coming back are raw, carriage returns and escape sequences and all, and they only mean anything to something that can interpret them. We do not interpret them. Writing that interpreter is writing a terminal emulator, and one already exists — NiceGUI ships xterm.js, so there is no CDN, no vendored blob, and nothing to fetch on a machine with no internet.

The bytes are also never decoded on our side. A multi-byte character split across a read boundary would be mangled; the browser reassembles the stream.

The output dialog waits for the human, not the command

`stream_pty()` finishing is not the end of the interaction — the person still has to *read* the thing. So the dialog returns when it is dismissed, not when the command exits.

Every action does `await _run_in_dialog(...)` and then `refresh()`, and refresh clears the container the dialog was created inside — deleting it. `up` and `pull` stream for long enough that nobody noticed. `logs` returns instantly, so the modal appeared and vanished before it could be read, and after a refresh there was nothing there at all.

The fix is to not hand control back to the code that clears things until the user is done looking. And then to `delete()` the dialog, because a closed dialog is still an element on the page, and a session spent starting and stopping things would otherwise accrete a pile of invisible modals still holding their output.

Refresh must not schedule itself inside the thing it clears

The other half of the same bug, and the one that actually blanked the page.

Every action refreshes the list afterwards, and `refresh()` schedules `render()` on a `ui.timer`. A NiceGUI event handler runs with the *clicked element's* slot active – and the button that was clicked lives in a card, inside the container. So the timer became a child of the container. `render()`'s first act is to clear that container, which deleted the timer whose callback was running, cancelling it half-done: cleared, and never repopulated. Close a dialog, and the page behind it was empty.

The first render works only by accident, because `page()` calls `refresh()` from the page slot, outside the container.

The timer is now pinned to `container.parent_slot`. Any code that reaches for `ui.timer` from inside a handler should ask the same question first: which slot is this actually landing in, and is something about to delete it?

Building a command is separate from running it

Every function that acts on a stack *returns* a `Command`; it does not execute one. That separation is what makes three things possible at once:

- the UI can show the exact command before and while it runs
- `--show` can print it instead of running it
- tests can assert on the command line with no docker daemon in sight

Most of the docker plugin's test suite runs on a machine with no docker installed, because the thing worth testing is *which command we built*.

There is no state that is ours

No database, no registry, no lockfile, no `enabled/` directory. A stack exists because a directory with a compose file exists. A stack is running because docker says it is running. We do not track anything, so we cannot drift out of sync with reality, and a user who edits the filesystem behind our back is not doing anything wrong – they are using the system exactly as intended.

This is why `--show` is safe on a production box and why `mkdir` is a supported way to add a service.

The project name is the directory name

We pass `-p <directory-name>` to compose, which is what compose would have defaulted to anyway. If we picked our own project name, SpiriConfig would be managing a *different set of containers* than the user gets from running `docker compose up` in that same directory, and they would never find ours. This one line is what keeps `spiriconfig docker up x` and a bare `docker compose up -d` interchangeable.

Compose files are text, not parsed YAML

Saving an edited compose file writes the user's text verbatim. We never round-trip it through a YAML parser, because that would silently eat their comments and reformat their file. We *validate* by asking `docker compose config` – the same authority that will have to accept the file later – and restore the original if it says no.

Advanced mode hides, it does not forbid

The web UI serves two audiences from one machine, so *advanced mode* filters what a page renders. It is tempting to reach for it as a permission system – "regular users cannot edit compose files" – and that would be a lie twice over.

The first lie is the usual one: the CLI still does everything, so a hidden button is not a removed capability.

The second is more immediate. **The toggle is self-service.** It is in the sidebar, it is labelled, and anyone can click it. A "regular user" who wants the Edit button turns advanced mode on and gets it, in the browser, without a shell. And below that, hiding an element is only a statement about what the page draws – the websocket underneath does not care what the browser chose to render.

Nothing is withheld from anyone, so there is nothing here to lean on.

Advanced mode is a preference, like a dark-mode switch. Nothing about it is a boundary, and it is not trying to be.

PAM is a login gate, not a permission model

`SPIRICONFIG_AUTH=pam` gates the UI behind a PAM login (see `spiriconfig.auth` and *configuration*), authenticating against the host's own accounts with no user list of ours. That is all it does. It answers one question – "does the machine trust this person enough to let them in?" – and nothing past the door.

So **everyone who gets in is equal.** They share the one process, running as whatever unix user SpiriConfig runs as, and can do everything the UI can do. PAM tells us *who* is at the keyboard; it does not, and is not trying to, tell us what they may touch once inside. There is no authorization layer, and none planned – this is a login gate, not a permissions system.

If we ever did want per-user permissions, they would belong to the OS, not to us: file modes, group membership, whether they are in the `docker` group – no role model, no ACL table, nothing of ours to get wrong or to keep

in sync. That is the same argument as *there is no state that is ours*, and it is the only permission design we would entertain. But that is a hypothetical, not a roadmap. Today the honest statement is the one above: authenticated users are undifferentiated.

An installed app is a symlink, and that is the whole app store

An *app store* is a git repo with one directory per app. Installing one is:

```
$ ln -s /var/lib/spiriconfig/stores/spiri-apps/whoami /srv/compose/whoami
```

That is not an implementation detail, it is the design. Everything an app store has to know is a question the symlink and the git repo behind it already answer:

Question	Answer
Where did this app come from?	`readlink`
Have I changed it?	`git status`
What did I change?	`git diff`
What did the store change?	`git diff HEAD @{upstream}`
Update it	`git merge`
Uninstall it	`rm` a symlink – which deletes nothing real

Note what is *not* on that list: anything of ours. There is no install manifest, no version database, no lockfile, no record of which commit an app came from. The symlink is the record, and git is the state – and git is a tool the user already has, already understands, and can drive without us when we are wrong.

It also means the docker plugin needed no changes at all. A symlink to a directory *is* a directory, so an installed app is just a stack: `spiriconfig docker up whoami` works on it, and so does `cd /srv/compose/whoami && docker compose up -d`. The app store and the thing that runs apps do not know about each other, which is why neither can drift out of sync with the other.

Versions are a label, not a mechanism

Nothing decides whether to update based on a version, because git can answer that question exactly: an app has an update if `git diff HEAD @{upstream} -- <app>/` says its files changed. That is what "update available" means, and it is precise even when the store bumps three images, rewrites a healthcheck, and touches a config file the maintainer forgot to bump a number for.

Which is the escape from a genuinely hard problem. "What version is this bag of containers?" has no good answer, so we do not need one to be right. The version string is *cosmetic*: `x-spiri-config-version` if a maintainer set one, and the date of the last commit touching the app if they did not. A store maintainer who never thinks about versioning still gets one that moves when the app does.

Updates are per-store, and that is on purpose

The symlinks all point into one working tree, which is at one commit, so "update whoami but not nextcloud" is not a state the repository can be in.

This sounds like a limitation and is closer to a feature. Pulling a store only rewrites files on disk: nothing is restarted, nothing is pulled from a registry, and no running container changes until someone runs `docker`

`compose up` on that stack. Per-app control still exists – it moved to up-time, which is where the user was going to make that decision anyway, and where they can see what they are about to restart.

An edit is a commit, so an update is a merge

Editing an installed app edits a file in the store's git working tree. So when the store moves on, an update is a three-way merge, and the user's changes and the store's changes are reconciled by the thing that is best in the world at reconciling them.

Before merging, we `git commit` whatever the user has edited. This is the step that makes an update non-destructive, and it is worth being explicit about why the alternatives are worse: `stash`, `reset`, and `checkout` all end with the user's work somewhere they will not think to look. Committing it puts the edit on the branch, so the merge has to *reconcile* with it. The worst case becomes a conflict they can see, rather than an edit that silently vanished – and it is what makes "undo the update" safe to offer, because their work survives the abort.

A conflict is left in the file with git's usual markers, and then something nice happens for free: **conflict markers are not valid YAML.** So `docker compose` refuses the file, and `Stack.write` – which validates with `docker compose config` before saving – refuses it too. A half-merged app cannot be started and cannot be saved over. We did not build that safety net; it fell out.

The one place git will *not* save us is finishing the merge. `git commit --all` stages unmerged files, which git reads as "the user resolved these" – so it will cheerfully commit a file still full of `<<<<<<<` markers and report a successful update. So `resolve` scans for markers itself and refuses while any remain. That guard is ours because there is no git flag that will do it.

Things the symlink design costs

Worth stating plainly, because both are real:

Relative bind mounts write into the store's checkout. A compose file with `./config:/config` resolves that path against the symlink, so docker creates `config/` inside the git clone. This is why "have I modified this app?" means *tracked* files only – otherwise every user would be told their apps were modified the moment they started one. Store authors should prefer named volumes.

Editing an installed app edits the store's copy of it. Two installs of the same app from the same store are the same files. If you want an app that is yours, `adopt` it: that replaces the symlink with a real copy, and the app leaves the store's orbit for good.

Two clever ideas that were rejected

Copy the app in, and record where it came from in a sidecar file. This is the obvious design, and it is worse in every particular: it invents a file format, makes SpiriConfig the keeper of provenance, and requires reimplementing three-way merge against a base version we would have to store somewhere. The symlink makes all of that git's problem, and git already solved it.

Make `/srv/compose` itself the clone, with sparse-checkout picking the installed apps. Seductive – install becomes `git sparse-checkout add whoami` – and fatally wrong. It makes the entire compose directory the property of one store, so the user's own hand-made projects become untracked files in somebody else's repository, `mkdir` stops being a supported way to add a service, and you can never have two stores. The user owns that tree. A symlink is a guest in it; a working tree is a landlord.

One seam for "whose setting is this?"

Advanced mode is per-person, and SpiriConfig currently has no people – so "person" means "browser", via a session cookie. Users are coming, and when they arrive the setting should follow the human, not the laptop.

Every preference therefore goes through *spiriconfig.preferences*, and nothing else touches browser storage. Adding user support is registering a different store, once; no plugin changes. The module-level store is a deliberate process-wide global – the alternative is threading a preferences object through every plugin's `page()`, which buys nothing and would be the very refactor the seam exists to avoid.

Plugins are entry points

Installing a package is what installs a plugin. This is not novel, and that is the appeal: it is how the rest of the Python ecosystem already works, so there is no bespoke plugin format, no plugin directory to manage, and nothing for us to keep in sync.

A broken plugin is logged and skipped, never fatal. The failure mode of a half-written plugin should be "my plugin is missing and the log says why", not "I can no longer administer this machine".

2.8.3 Things we deliberately do not do

An "enabled" state. An earlier design had nginx-style `available/` + `enabled/` symlinks. It was dropped: docker already knows whether a stack is running, so a second source of truth could only ever disagree with the first. Up and down is the whole lifecycle.

Creating or deleting project directories. The user owns that tree. We read it.

The *app store* does not bend this rule, it is shaped by it. An install creates a *symlink*, and an uninstall removes one; the app's actual files live in the store's checkout the whole time, and are still there afterwards. If installing meant `cp -r` and uninstalling meant `rm -rf`, we would be deleting directories full of the user's data on their behalf, and this line would have had to go. It did not, and the design is better for having had to obey it.

Anything clever with the docker socket. If a thing cannot be expressed as a command line, it does not belong in a plugin – because it could not be shown to the user, and they could not do it themselves.

2.9 API reference

2.9.1 Plugins

The plugin interface, and its discovery.

A plugin is a class that subclasses *Plugin* and is registered under the `spiriconfig.plugins` entry point group:

```
[project.entry-points."spiriconfig.plugins"]
myplugin = "my_package:MyPlugin"
```

Installing the distribution makes the plugin available; uninstalling it removes the plugin. There is no plugin registry to edit and no enable/disable state for us to keep in sync with reality.

A plugin supplies two faces onto the same functionality:

- *Plugin.cli()* – a Typer app, mounted as `spiriconfig <name> ...`

- *Plugin.page()* – a NiceGUI page body, mounted at `/<name>`

Both are optional, but a plugin that offers a web page and no CLI is a bug in spirit: the UI must not be the only way to do something.

class spiriconfig.plugins.**Plugin**

Bases: ABC

Base class for SpiriConfig plugins.

name: str

Short identifier. Used for the CLI subcommand, the URL, and the log field.

title: str

Human-readable name, shown in the web UI.

description: str = ''

One-line summary, shown in the CLI help and on the web UI's index.

icon: str = 'extension'

Material icon name, shown beside the title in the web UI's sidebar.

advanced: bool = False

Whether this plugin is only *shown* in advanced mode.

For a plugin that is a developer's tool rather than an operator's – the terminal is the obvious one. It keeps the sidebar about the things a normal user came here to do, without the plugin having to wrap its own page in *spiriconfig.advanced.only()* and hope it remembered every entry point.

Shown, and nothing more. The page is still routed and the CLI still works with advanced mode off, exactly as *spiriconfig.advanced* says: this is a filter on the UI, not a permission, and a plugin that sets this flag has not been secured by it. If a plugin must not be *reachable*, do not install it.

cli()

Return a Typer app to mount at `spiriconfig <name>`, or None.

Return type
Typer | None

page()

Render this plugin's NiceGUI page body.

Called inside a page route, so it may use `ui.*` freely. It must not be the only way to reach the plugin's functionality – anything doable here must also be doable from *cli()* or from the underlying tool.

Return type
None

async on_startup()

Run once when the web server starts. Default: nothing.

Registered with the server's startup hook, for work that belongs to the process rather than to a page load – warming a cache, a first fetch. Do not block: the server does not begin serving until every plugin's `on_startup` has returned, so anything slow (or reaching the network) should launch a background task and return, not await it here.

Only fires in the web path (`spiriconfig web`); the CLI never starts a server, so it never calls this.

Return type
None

property has_page: bool

Whether this plugin overrides *page()*.

spiriconfig.plugins.**discover**()

Load every installed plugin, sorted by name.

Broken plugins are logged and skipped rather than raising.

Return type
list[Plugin]

2.9.2 Advanced mode

Advanced mode: a filter on what the web UI *shows*.

Advanced mode hides clutter. It is not a permission system, and it must never be used as one – a hidden button is still a reachable capability, and the CLI does everything regardless of what the UI is currently showing. That is deliberate: the CLI is the escape hatch that makes progressive enhancement true, so gating it would undercut the whole project. If you ever need "this person may not restart containers", that is authorisation, and it belongs in front of the *command*, not in front of the button.

So: advanced mode decides what a page renders. Nothing else.

Plugins use it like this:

```
from spiriconfig import advanced

ui.button("Up", on_click=...)          # everyone sees this

with advanced.only():
    ui.button("Edit", on_click=...)    # only developers see this
```

Elements inside *only()* are *bound* to the setting rather than conditionally created, so flipping the toggle shows and hides them instantly, with no page rebuild and no lost state in whatever the user was doing.

A container plugin (a separate process behind the reverse proxy, see `spiriconfig.proxy` and `docs/container-plugins.md`) cannot call *enabled()* – it does not share this interpreter. What it *can* do is read *BROWSER_STORAGE_KEY* out of `localStorage` and listen for the browser's own `storage` event, because the plugin is proxied onto the same origin as the shell and `localStorage` is shared across same-origin documents, iframe or not. This module keeps that key mirrored on every change, fire-and-forget, so a plugin gets a live signal with no polling and no change to the proxy's wire contract:

```
window.addEventListener('storage', event => {
    if (event.key === 'spiriconfig-developer-mode') { ... }
});
```

spiriconfig.advanced.**PREFERENCE_KEY = 'advanced'**

Key this setting is stored under, in whatever store *preferences()* resolves to.

spiriconfig.advanced.**BROWSER_STORAGE_KEY = 'spiriconfig-developer-mode'**

`localStorage` key mirroring the current value, for container plugins to read – see the module docstring. Not a security boundary: anything in `localStorage` is as trustworthy as the browser it's running in, same as advanced mode itself. "developer-mode" rather than "advanced" – this is the string a plugin author reads and copies, so it should say what the UI now says.

class spiriconfig.advanced.**AdvancedState**(*enabled=False*)

Bases: `object`

The live setting for one connected client.

A bindable dataclass, so that assigning to *enabled* immediately propagates to every element bound to it.

Parameters

enabled (*bool*)

spiriconfig.advanced.**state**()

Return the advanced-mode state for the client being served right now.

Seeded on first access from the person's stored preference, falling back to `SPIRICONFIG_ADVANCED` – so a developer image can ship with advanced mode on by default, and a customer image with it off, from the same code.

The live object is held per connection; the *durable* value lives in the preference store, which is the thing that will become per-user.

Return type

AdvancedState

spiriconfig.advanced.**enabled**()

Whether advanced mode is on for the client being served right now.

Return type

bool

spiriconfig.advanced.**set_enabled**(*value*)

Turn advanced mode on or off, and remember the choice.

The live state updates first so the UI responds even if persistence fails – a preference we could not save is a much smaller problem than a toggle that appears not to work.

Parameters

value (*bool*)

Return type
None

spiriconfig.advanced.**mark**(*element*)

Show `element` only in advanced mode. Returns it, so it chains.

Also makes it *look* advanced, which is the same act: an element cannot be advanced-only and yet fail to say so, because this is the only way to make it advanced-only in the first place.

How it says so depends on what it is. A button takes the purple as its Quasar `color` prop – purple lettering on a flat button, a purple face on a solid one – because Quasar's own stylesheet sets `outline: 0` on `.q-btn`, and a ring drawn on a button is a fight with the framework that we would be re-fighting at every upgrade. Everything else gets the dashed ring.

Parameters
element (*Element*)

Return type
Element

spiriconfig.advanced.**only**()

Show everything created inside this block only in advanced mode.

Binds the elements that appear in the current slot, rather than wrapping them in a container: a wrapper element would sit inside the parent's flex or grid layout and quietly change how the visible siblings are arranged.

Return type
Iterator[None]

spiriconfig.advanced.**toggle**()

A switch for advanced mode. Always visible – it is the way back.

Purple when it is on, and grey when it is off, wearing the same colour as the ring around everything it reveals: the switch is the legend for the marks.

Return type
Switch

2.9.3 Preferences

Where a person's UI preferences are stored.

This module exists to be *one* seam. Today SpiriConfig has no concept of a user, so a preference belongs to a browser: NiceGUI keeps a session cookie, and we hang the preference off that. When real users arrive, a preference should follow the person across their devices instead.

Rather than sprinkling `app.storage.user` through the codebase and having to find every call site later, everything goes through *`preferences()`*. Adding user support then means registering a different store here, once:

```
from spiriconfig import preferences

```

(continues on next page)

(continued from previous page)

```
class DatabasePreferences:
    def __init__(self, user_id: str) -> None:
        self.user_id = user_id

    def get(self, key, default):
        return db.fetch_preference(self.user_id, key, default)

    def set(self, key, value) -> None:
        db.store_preference(self.user_id, key, value)

preferences.use(lambda: DatabasePreferences(current_user().id))
```

Nothing else changes. Plugins call *spiriconfig.advanced.enabled()*; they do not know, and must not care, whether that preference is keyed on a cookie or on a person.

class spiriconfig.preferences.**PreferenceStore**(**args*, ***kwargs*)

Bases: Protocol

Somewhere to read and write one person's UI preferences.

get(*key*, *default*)

Return the stored value for `key`, or `default`.

Parameters

- **key** (*str*)
- **default** (*Any*)

Return type
Any

set(*key*, *value*)

Store `value` under `key`.

Parameters

- **key** (*str*)
- **value** (*Any*)

Return type
None

class spiriconfig.preferences.**BrowserPreferences**

Bases: object

Preferences kept against NiceGUI's per-browser session cookie.

The default store, and the correct one while there are no users: it is the finest-grained identity we actually have. It survives a reload and is shared across a browser's tabs, but not across devices – which is exactly the limitation that user support will lift.

Requires a storage secret (see *storage_secret*), because the cookie that identifies the browser is signed.

spiriconfig.preferences.**use**(*factory*)

Register the store that *preferences()* will return from now on.

`factory` is called per access rather than once, so it can resolve the *current* person from request context – which is what a user system needs.

Parameters

factory (*Callable[[], PreferenceStore]*)

Return type

None

spiriconfig.preferences.**reset**()

Restore the default per-browser store. Mostly for tests.

Return type

None

spiriconfig.preferences.**preferences**()

Return the preference store for whoever is being served right now.

Return type

PreferenceStore

2.9.4 Commands

Running external commands.

Everything SpiriConfig does to the system is done by shelling out to the tool a human would have used. This module is the only place that spawns processes, and *Command* is deliberately printable: the string it renders is the exact line a user can paste into a shell to do the same thing by hand.

That is the whole point of the design. A plugin should never reach for a Python API when a command line exists, because then the UI can do something the user cannot reproduce or audit.

class spiriconfig.commands.**Command**(*argv*, *cwd=None*, *env=<factory>*)

Bases: object

An external command, plus the context needed to reproduce it by hand.

Parameters

- **argv** (*Sequence[str]*)
- **cwd** (*Path | None*)
- **env** (*dict[str, str]*)

class spiriconfig.commands.**Result**(*command*, *returncode*, *stdout*, *stderr*)

Bases: object

The outcome of a *Command* that ran to completion.

Parameters

- **command** (Command)
- **returncode** (*int*)
- **stdout** (*str*)
- **stderr** (*str*)

check()

Return self, or raise *CommandError* if the command failed.

Return type
Result

exception spiriconfig.commands.**CommandError**(*result*)

Bases: RuntimeError

A command exited non-zero.

Parameters
result (Result)

Return type
None

spiriconfig.commands.**run**(*command*, *, *timeout=60.0*, *input=None*, *log=<loguru.logger handlers=[(id=0, level=10, sink=<stderr>)]>*)

Run `command` to completion and capture its output.

Use this for short, quick commands. Anything that streams output or may run for a long time (`up`, `pull`, `logs -f`) should use *stream()*, so the user sees progress instead of a spinner.

The command line is logged at DEBUG rather than INFO. Captured commands are overwhelmingly read-only queries – statuses, listings – and logging each one at INFO buries the commands that actually changed something.

`input` is written to the process's stdin and, crucially, is **never logged** – unlike `Command.argv`, which is. That asymmetry is the point: it is the one channel for a secret a command must be told but that must not appear in the log or in the copy-pasteable line. `docker login --password-stdin` and `git credential approve` are exactly why it exists.

Pass `log` to attribute the command to a plugin, e.g. `log=logger.bind(plugin="docker")`.

Parameters

- **command** (Command)
- **timeout** (*float | None*)
- **input** (*str | None*)
- **log** (*Logger*)

Return type
Result

class spiriconfig.commands.**PtySession**(*command*, ***, *rows=30*, *columns=120*, *log=<loguru.logger handlers=[(id=0, level=10, sink=<stderr>)]>*)

Bases: object

A long-lived process on a pty, with the *input* end still connected.

stream_pty() runs a command and lets you watch it. This lets you talk back to it, which is a different thing and needs a different shape: there is no natural end to wait for, the size can change while it runs, and something has to notice when the person at the other end walks away.

So it is a session rather than a generator, and it is the caller's job to end it. In the web UI the caller is a page, and the page ends the session when the browser disconnects – see spiriconfig_terminal.web.

Parameters

- **command** (Command)
- **rows** (*int*)
- **columns** (*int*)
- **log** (*Logger*)

property running: bool

Whether the process is alive and its pty is still open.

async start()

Spawn the process on a new pty and begin reading its output.

Spawned with subprocess.Popen rather than asyncio's create_subprocess_exec, which is the odd one out in this module and is not a matter of taste.

The child needs _take_controlling_terminal() to run between the fork and the exec, and the only way to ask for that is preexec_fn – which **uvloop refuses to implement**. It raises SubprocessError: Exception occurred in preexec_fn rather than run it. That matters here and nowhere else in this file, because NiceGUI serves on uvloop: this code would work in every test and then fail on the only loop that is ever going to run it. Popen forks the child itself, so no event loop gets a say, and it works the same under both.

Return type
None

async output()

Yield raw output bytes until the process exits or the session is closed.

Raw, exactly as *stream_pty()* yields them, and for the same reason: the escape sequences *are* the output, and xterm.js is the thing that knows what they mean.

Return type
AsyncIterator[bytes]

write(*data*)

Send keystrokes to the process, as though they had been typed.

Silently does nothing once the session is over. A key pressed in a dead terminal is not an error worth raising into the page – the person can already see that the shell has exited, because it said so.

Parameters
data (*bytes | str*)

Return type
None

resize(*rows*, *columns*)

Tell the process its terminal is now this big.

Recorded even when there is no pty yet, so that a size arriving from the browser *before* the process starts is not lost – it becomes the size the process is born with, which is the whole point of asking.

Parameters

- **rows** (*int*)
- **columns** (*int*)

Return type
None

close()

End the session: hang up the terminal, and let the process notice.

Closing the master end is a hangup, and the kernel does the rest – SIGHUP to the session leader and to the pty's foreground process group. That is precisely what happens when a person closes a terminal window, so a shell needs no special handling to cope with it, and neither does whatever the shell was running: *ping* dies here for the same reason it dies there.

Return type
None

async wait()

Wait for the process to exit, and reap it. Returns its exit code.

Negative when a signal ended it, which is the ordinary case here: hanging up the terminal kills the shell with SIGHUP, so a closed browser tab comes back as `-1` rather than as a failure.

`Popen.wait` blocks, so it goes to a thread. Blocking the event loop here would freeze the UI of every other person connected – and the thing we are waiting for is a shell, which might not exit for hours.

Return type
int

async spiriconfig.commands.**stream_pty**(*command*, *, *log=<loguru.logger handlers=[(id=0, level=10, sink=<stderr>)]>*, *rows=30*, *columns=120*)

Run `command` attached to a pseudo-terminal, yielding raw output bytes.

The difference between this and *stream()* is not cosmetic, and it is not really about colour either.

Docker asks whether its output is going to a terminal, and changes what it *says* based on the answer. Through a pipe it gives up and emits a flat transcript; on a tty it draws progress bars, redraws layer

download status in place, and uses colour to separate services. So piping the output does not give us a plain version of the same information – it gives us a program that decided we were not worth talking to properly. *docker compose pull* is the obvious victim: on a pipe it is a wall of "Pulling", on a tty it is a live picture of what is happening.

So we give it a terminal. The bytes that come back are raw, carriage returns and escape sequences and all, and they are only meaningful to something that can interpret them – which is why they go to xterm.js in the browser rather than to `ui.log`. Interpreting them ourselves would mean writing a terminal emulator, which is a thing that already exists.

Yields `bytes`, deliberately: decoding here would mean splitting on character boundaries we do not control, and a multi-byte character straddling a read boundary would be mangled. The browser reassembles the stream.

Parameters

- **command** (Command)
- **log** (*Logger*)
- **rows** (*int*)
- **columns** (*int*)

Return type
AsyncIterator[bytes]

async spiriconfig.commands.**stream**(*command*, *, *log=<loguru.logger handlers=[(id=0, level=10, sink=<stderr>)]>*)

Run `command`, yielding output lines as they arrive.

stderr is folded into stdout because that is what the user would see in a terminal, and docker compose writes most of its progress to stderr. The final line yielded is a synthetic marker when the command fails, so callers that only render lines still surface the failure.

Unlike *run()*, the command line is logged at INFO: these are the long, state-changing commands, and the log is the record of what we did to the user's machine.

Parameters

- **command** (Command)
- **log** (*Logger*)

Return type
AsyncIterator[str]

2.9.5 Terminal

The terminal widgets: one to watch a command in, and one to type into.

SpiriConfig runs the command a human would have run, so the honest place to show its output is the thing a human would have watched it in: a terminal. This module is the xterm.js configuration, shared so that every plugin's output looks the same and none of them has to think about it.

There are two of them, and the difference between them is not decoration:

terminal() is a **transcript**. A command was decided on, it ran, and this shows what it said. It takes no input, and its size is fixed rather than fitted to the browser window – fitting means asking the browser how many columns it ended up with and waiting for the answer before a single byte can be read, and it means the same command wraps differently on a laptop and on a phone. A fixed 120x30 is what the command sees, always, and the box scrolls if the window is smaller.

interactive() is a **terminal**, in the sense the word usually has. There is a shell at the other end, so every one of those choices inverts: it takes input, it blinks a cursor to say so, and it *does* fit the window, because a person resizing their browser expects vim to still fill it. The round trip we refuse to pay for a transcript is exactly what an interactive session is worth paying for, since there is nobody waiting on the first byte – the shell is waiting on them.

pump() is what makes an *interactive()* pane actually live: it settles the size, starts the process, and copies bytes between the two until one end gives up. It lives here rather than in any one plugin because there is now more than one thing on the far end of an interactive terminal – a login shell on the terminal page, and `docker compose exec` or `attach` on the apps page – and every one of them has to get the hangup right or it leaks a process per browser tab.

spiriconfig.terminal.**TERMINAL_ROWS = 30**

The size of the pseudo-terminal commands run in, and of the widget that shows them. These must agree: the pty tells the program how wide it is, and the program wraps and draws progress bars to that width. If the widget were narrower, docker's careful in-place redraws would land in the wrong columns.

spiriconfig.terminal.**interactive**(** (Keyword-only parameters separator (PEP 3102))*, *on_data*, *on_resize*)

An xterm.js pane wired for two-way traffic with a *PtySession*.

Both handlers are required, because a terminal missing either one is broken in a way that looks like a hang. Without `on_data` the keystrokes go nowhere and the shell appears frozen; without `on_resize` the pty keeps insisting the window is its original size, and every full-screen program draws to the wrong one.

The caller still has to call `fit()` once the browser has laid the element out, and hand the resulting size to the session before starting it.

Parameters

- **on_data** (*Callable[[XtermDataEventArguments], Any] | Callable[[], Any]*)
- **on_resize** (*Callable[[XtermResizeEventArguments], Any] | Callable[[], Any]*)

Return type
Xterm

async spiriconfig.terminal.**pump**(*session*, *view*, *client*, ***, *what='the command'*, *log=<loguru.logger handlers=[(id=0, level=10, sink=<stderr>)]>*)

Start `session` at `view`'s size and copy it into `view` until it ends.

The size is settled before the process is spawned, not after. A shell that is born believing it has 80 columns and is corrected a moment later has already printed its prompt at the wrong width, and anything it drew

is smeared – so we pay for the round trip to the browser first, once, and let it open its eyes at the size it is actually going to live at.

`client` is taken as an argument rather than read from `nicegui.context` because this is called from a task, and the caller is the one that still definitely knows which browser it is serving. It is what the session is hung up on when that browser goes away – and something has to be, because nobody sends us a goodbye when a laptop lid comes down. Without it every visit leaks a process, and a device that has been debugged a few times is quietly hosting a dozen of them.

`what` names the thing in the closing line: "the shell exited with code 0".

Parameters

- **session** (PtySession)
- **view** (*ui.xterm*)
- **client** (*Client*)
- **what** (*str*)
- **log** (*Logger*)

Return type
None

spiriconfig.terminal.**terminal**()

An xterm.js pane, sized to match *spiriconfig.commands.stream_pty()*.

Write raw `bytes` straight from `stream_pty` into it. Do not decode them and do not clean them up: the carriage returns and escape sequences are the output, and xterm.js is the thing that knows what they mean.

Return type
Xterm

2.9.6 Configuration

Configuration, read from the environment.

SpiriConfig is configured entirely through environment variables. There is no config file format to learn, and no config file for us to rewrite behind the user's back – a service manager, a shell, or a container runtime can all supply settings the same way.

Core settings are prefixed `SPIRICONFIG_`. Plugins get their own prefix (see *Plugin*) so that their settings are namespaced, e.g. `SPIRICONFIG_DOCKER_`.

class spiriconfig.config.**Settings**(*_case_sensitive=None*, *_nested_model_default_partial_update=None*, *_env_prefix=None*, *_env_prefix_target=None*, *_env_file=PosixPath('.')*, *_env_file_encoding=None*, *_env_ignore_empty=None*, *_env_nested_delimiter=None*, *_env_nested_max_split=None*, *_env_parse_none_str=None*, *_env_parse_enums=None*, *_cli_prog_name=None*, *_cli_parse_args=None*, *_cli_settings_source=None*, *_cli_parse_none_str=None*, *_cli_hide_none_type=None*, *_cli_avoid_json=None*, *_cli_enforce_required=None*, *_cli_use_class_docs_for_groups=None*, *_cli_exit_on_error=None*, *_cli_prefix=None*, *_cli_flag_prefix_char=None*, *_cli_implicit_flags=None*, *_cli_ignore_unknown_args=None*, *_cli_kebab_case=None*, *_cli_shortcuts=None*, *_secrets_dir=None*, *_build_sources=None*, ***, *host='127.0.0.1'*, *port=8337*, *log_level='INFO'*, *plugin_discovery_interval=5.0*, *log_file=None*, *advanced=False*, *storage_secret=None*, *session_cookie='session'*, *tls='auto'*, *tls_cert=None*, *tls_key=None*, *auth='pam'*, *auth_service='login'*, *auth_group='sudo'*)

Bases: `BaseSettings`

Core settings, shared by the CLI and the web UI.

Parameters

- **_case_sensitive** (*bool | None*)
- **_nested_model_default_partial_update** (*bool | None*)
- **_env_prefix** (*str | None*)
- **_env_prefix_target** (*EnvPrefixTarget | None*)
- **_env_file** (*DotenvType | None*)
- **_env_file_encoding** (*str | None*)
- **_env_ignore_empty** (*bool | None*)
- **_env_nested_delimiter** (*str | None*)
- **_env_nested_max_split** (*int | None*)
- **_env_parse_none_str** (*str | None*)
- **_env_parse_enums** (*bool | None*)
- **_cli_prog_name** (*str | None*)
- **_cli_parse_args** (*bool | list[str] | tuple[str, ...] | None*)
- **_cli_settings_source** (*CliSettingsSource[Any] | None*)
- **_cli_parse_none_str** (*str | None*)

- **_cli_hide_none_type** (*bool | None*)
- **_cli_avoid_json** (*bool | None*)
- **_cli_enforce_required** (*bool | None*)
- **_cli_use_class_docs_for_groups** (*bool | None*)
- **_cli_exit_on_error** (*bool | None*)
- **_cli_prefix** (*str | None*)
- **_cli_flag_prefix_char** (*str | None*)
- **_cli_implicit_flags** (*bool | Literal['dual', 'toggle'] | None*)
- **_cli_ignore_unknown_args** (*bool | None*)
- **_cli_kebab_case** (*bool | Literal['all', 'no_enums'] | None*)
- **_cli_shortcuts** (*Mapping[str, str | list[str]] | None*)
- **_secrets_dir** (*PathType | None*)
- **_build_sources** (*tuple[tuple[PydanticBaseSettingsSource, ...], dict[str, Any]] | None*)
- **host** (*str*)
- **port** (*int*)
- **log_level** (*str*)
- **plugin_discovery_interval** (*float*)
- **log_file** (*str | None*)
- **advanced** (*bool*)
- **storage_secret** (*str | None*)
- **session_cookie** (*str*)
- **tls** (*Literal['auto', 'off']*)
- **tls_cert** (*str | None*)
- **tls_key** (*str | None*)
- **auth** (*Literal['none', 'pam']*)
- **auth_service** (*str*)
- **auth_group** (*str*)

model_config = {'arbitrary_types_allowed': True, 'case_sensitive': False, 'cli_avoid_json': False, 'cli_enforce_required': False, 'cli_exit_on_error': True, 'cli_flag_prefix_char': '-', 'cli_hide_none_type': False, 'cli_ignore_unknown_args': False, 'cli_implicit_flags': False, 'cli_kebab_case': False, 'cli_parse_args': None, 'cli_parse_none_str': None, 'cli_prefix': '', 'cli_prog_name': None, 'cli_shortcuts': None, 'cli_use_class_docs_for_groups': False, 'enable_decoding': True, 'env_file': '.env', 'env_file_encoding': None, 'env_ignore_empty': False, 'env_nested_delimiter': None, 'env_nested_max_split': None, 'env_parse_enums': None, 'env_parse_none_str': None, 'env_prefix': 'SPIRICONFIG_', 'env_prefix_target': 'variable', 'extra': 'ignore', 'json_file': None, 'json_file_encoding': None, 'nested_model_default_partial_update': False, 'protected_namespaces': ('model_validate', 'model_dump', 'settings_customise_sources'), 'secrets_dir': None, 'toml_file': None, 'validate_default': True, 'yaml_config_section': None, 'yaml_file': None, 'yaml_file_encoding': None}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

host: str

Address the web UI binds to. `SPIRICONFIG_HOST`.

Defaults to loopback so a fresh install is not exposed to the network. Set `SPIRICONFIG_HOST=0.0.0.0` to bind all interfaces once you have decided the UI should be reachable from elsewhere.

port: int

Port the web UI binds to. `SPIRICONFIG_PORT`.

log_level: str

Minimum log level. `SPIRICONFIG_LOG_LEVEL`.

plugin_discovery_interval: float

Seconds between rescans for proxied (container) plugins. `SPIRICONFIG_PLUGIN_DISCOVERY_INTERVAL`.

Out-of-process plugins are discovered from docker labels (see `spiriconfig.discovery`); this is how often that view is refreshed, so an app installed or stopped while SpiriConfig runs appears or disappears on its own. Set to `0` to scan once at startup and never again – for an appliance whose plugin set is fixed at boot, where the repeated `docker` calls buy nothing.

log_file: str | None

Optional file to log to, in addition to stderr. `SPIRICONFIG_LOG_FILE`.

advanced: bool

Default for advanced mode, for someone who has not chosen. `SPIRICONFIG_ADVANCED`.

Only a *default*: each person can toggle it in the UI, and their choice wins from then on. Ship a developer image with this on and a customer image with it off, from the same code.

Advanced mode hides features from the web UI. It is not a permission boundary – see *spiriconfig.advanced*.

storage_secret: str | None

Signs the cookie that identifies a browser. SPIRICONFIG_STORAGE_SECRET.

Set this to something secret and stable, or per-person settings (advanced mode today, anything user-scoped later) reset every time the process restarts, because the cookie they were keyed on can no longer be verified.

session_cookie: str

Name of that cookie. SPIRICONFIG_SESSION_COOKIE.

Only matters when one SpiriConfig is proxied inside another (a robot's UI inside the SDK's, at /plugin/<name>/). The two then share an origin, so with the same cookie name each login overwrites the other's. Give the inner one its own name, e.g. spiriconfig-mu-1.

tls: Literal['auto', 'off']

Whether the web UI serves HTTPS, and how. SPIRICONFIG_TLS.

auto (the default) serves a **self-signed** cert whenever the UI is bound off loopback and no cert is provided, so an exposed appliance is never sending the PAM password over plain HTTP. That stops a passive eavesdropper; it does *not* stop an active MITM (the browser warns, the operator clicks through) – for that, provide a cert the browser can validate via *tls_cert*/*tls_key*. A loopback dev session stays on plain HTTP, where a cert warning is pure friction.

off serves plain HTTP regardless: the escape hatch for a deployment whose own reverse proxy terminates TLS in front of us, where encrypting again is wasted work. See spiriconfig.tls.

tls_cert: str | None

Path to a TLS certificate to serve. SPIRICONFIG_TLS_CERT.

Set this and *tls_key* together to serve a cert of your own instead of a generated self-signed one. This is the only path that resists an active MITM: a per-device cert from a CA the fleet trusts is one the browser *validates*, so an attacker's substitute is rejected rather than clicked through. HSTS is enabled only on this path, never for self-signed.

tls_key: str | None

Path to the private key for *tls_cert*. SPIRICONFIG_TLS_KEY.

Required with *tls_cert*; one without the other is ignored with a warning.

auth: Literal['none', 'pam']

Whether the web UI requires a login, and how. SPIRICONFIG_AUTH.

pam (the default) puts a login in front of every page, authenticating against the host's PAM stack – see spiriconfig.auth. none serves the UI to anyone who can reach the port, which is only appropriate for a checkout on loopback with no other users on the box.

On by default: a login gate is the right default even for a single-operator box, and a developer who wants a passwordless loopback session can opt out explicitly.

auth_service: **str**

PAM service to authenticate against. `SPIRICONFIG_AUTH_SERVICE`.

The name of a file under `/etc/pam.d/`. `login` exists on essentially every system and reads the normal password stack, so it is the default. A deployment that wants its own policy ships `/etc/pam.d/spiriconfig` and sets this to `spiriconfig`. Only consulted when *`auth`* is `pam`.

auth_group: **str**

Group whose members may log in, *when SpiriConfig runs as root*. `SPIRICONFIG_AUTH_GROUP`.

Running as root, PAM can verify any account's password, so we would otherwise let every system user (`nobody`, service accounts, …) into the admin UI. Membership of this group is the gate. `sudo` on Debian/Ubuntu (the default), `wheel` on Arch/RHEL – set it to match the box.

Ignored when SpiriConfig does not run as root: PAM can then only verify the one account the process runs as, so there is nothing to gate. See `spiriconfig.auth.authenticate()`.

`spiriconfig.config.`**`settings`**`()`

Load core settings from the environment.

Not cached: tests and long-lived processes should be able to observe an changed environment without reaching into a module global.

Return type
Settings

2.9.7 The docker plugin

Compose projects, and the docker commands that act on them.

A *stack* is one subdirectory of the configured compose directory that contains a compose file. That is the entire model. It maps exactly onto how `docker compose` already works, so a user who has never run SpiriConfig can drop a directory in, and a user who never opens the web UI can `cd` into one and run `docker compose up -d` themselves.

Every function here that acts on a stack returns a *`Command`* rather than running it. Building and running are kept separate so the UI can show the user what it is about to do, and so tests can assert on the command line without a docker daemon anywhere in sight.

`spiriconfig_docker.stacks.`**`COMPOSE_FILENAMES = ('compose.yaml', 'compose.yml', 'docker-compose.yaml', 'docker-compose.yml')`**

Compose file names, in the precedence order docker compose itself uses.

`spiriconfig_docker.stacks.`**`DEFAULT_EXEC_COMMAND = '/bin/sh'`**

What *`Stack.exec()`* runs when nobody says otherwise. See the method for why it is `sh` and not `bash`.

`spiriconfig_docker.stacks.`**`DEV_OVERRIDE_FILENAMES = ('compose.dev.yaml', 'compose.dev.yml', 'docker-compose.dev.yaml', 'docker-compose.dev.yml')`**

Dev-override compose files, one per base name. These are *overrides* layered on top of the base file (`docker compose -f compose.yaml -f compose.dev.yaml`), not standalone stacks – exactly what the app template's `compose.dev.yaml` is written to be: build from source, mount it, live-reload. Optional, and only ever reached in advanced mode; a stack without one simply cannot be brought up in dev.

spiriconfig_docker.stacks.**NEW_COMPOSE_TEMPLATE = 'services:\n whoami:\n image: traefik/whoami\n ports:\n - "8080:80"\n'**

What a freshly created compose file starts as, when the user gives us no text of their own. Deliberately a working stack and not an empty file: `docker compose config` rejects a file with no services, so a blank template would fail the very validation *create()* runs, and the first thing a new user saw would be an error. `whoami` is tiny, real, and starts.

class spiriconfig_docker.stacks.**Stack**(*name*, *path*, *compose_file*, *settings*, *dev_override=None*)

Bases: `object`

One compose project on disk.

Parameters

- **name** (*str*)
- **path** (*Path*)
- **compose_file** (*Path*)
- **settings** (DockerSettings)
- **dev_override** (*Path | None*)

name: str

The directory name, which is also the compose project name.

path: Path

The project directory.

compose_file: Path

The compose file within *path*.

dev_override: Path | None

A dev-override compose file to layer on *compose_file* in dev mode.

`None` for the common case of a stack that ships no such file. See *DEV_OVERRIDE_FILENAMES* and *up()*.

up(*, *dev=False*)

Create and start the stack, in the background.

With `dev=True` and a dev-override file present, layer it on top of the base compose (`-f compose.yaml -f compose.dev.yaml`) so the stack runs the app author's development variant – build from source, mount it, live-reload. That variant also gets `--build`: compose only builds an image that does not exist yet, so without it a second dev launch would run yesterday's source. Without an override the flag does nothing, so a plain stack still comes up.

Parameters

dev (*bool*)

Return type

Command

down()

Stop and remove the stack's containers.

Return type
Command

restart()

Restart the stack's containers.

Return type
Command

pull()

Pull the stack's images.

Return type
Command

logs(*, *follow=False*, *tail=200*)

Show the stack's logs.

Parameters

- **follow** (*bool*)
- **tail** (*int*)

Return type
Command

ps()

List the stack's containers, as JSON.

Return type
Command

exec(*service*, *command=()*, *, *pidfile=None*)

Run a command inside a service's *running* container.

The command defaults to *DEFAULT_EXEC_COMMAND*, which is a shell, because "give me a prompt in there" is what people overwhelmingly want. It is only a default: this is `docker compose exec`, so anything the image can run is fair game, and the UI hands the user a box to say so.

`/bin/sh` rather than `bash`: sh is the one a container is actually likely to have. Alpine – which half the images on a device are built from – ships busybox and no bash at all, and a button that dies with "executable file not found" on half the apps is worse than a plainer shell that works on all of them. Somebody who wants bash types bash.

No TTY flag, because there is none to pass: unlike `docker exec`, which wants `-it`, `docker compose exec` allocates a TTY *by default* and the only related flag it has is `-T`, to turn one off. The shell on the far side gets a terminal without our asking for one.

`pidfile` is what the web terminal passes, and it is the whole of our answer to *hangup()* – see there for why it has to exist at all. The command line stops being the one a person would type

when it is set, which is precisely why it is *optional*: the CLI and the line shown on the page are built without it, and only the session that has a browser tab to lose is wrapped.

Parameters

- **service** (*str*)
- **command** (*Sequence[str]*)
- **pidfile** (*str | None*)

Return type
Command

hangup(*service*, *pidfile*)

Kill what a `pidfile`'d *exec()* left running in the container.

This exists because **docker will not do it for us**. That is a strong claim about somebody else's software, so it is worth writing down what was actually measured, because every plausible easier answer is wrong:

- It is not a missing `-it`. Unlike `docker exec`, `docker compose exec` allocates a TTY *by default*; the only flag it has is `-T`, to turn one off. Passing `-it` to a plain `docker exec` leaks identically.
- It is not a pty we forgot to close. With the client verifiably reaped and nothing on the host holding the slave device open, the process inside the container carries on regardless.
- It is not a signal we failed to send. SIGHUP kills the client and leaks; SIGKILL leaks; SIGTERM and SIGINT do not even *kill the client*, which sits in raw mode forwarding them as bytes. There is no polite signal that makes it clean up, because there is nothing listening for one.

The reason is that an exec'd process is not the client's child. It is parented inside the container, and the daemon does not tear an exec down when the hijacked API stream drops – so the shell gets no EOF, no SIGHUP, and no timeout. It simply carries on, forever, until the container stops. The docker API can start an exec and inspect an exec, and offers no way whatsoever to kill one; hence a pidfile, which is the shape everybody who has hit this ends up at. Twelve years and counting:

- https://github.com/moby/moby/issues/9098 (open since 2014: killing the `docker exec` client does not terminate the spawned process)
- https://github.com/moby/moby/issues/29700 (the orphan, reparented to PPID 0, when the client disconnects)
- https://github.com/moby/moby/issues/35703 (the request for a `docker exec kill` that would make this method unnecessary)

Nobody meets this at a real terminal, because at a real terminal you type `exit` – and a shell that exits is clean. A browser tab has no `exit`; closing it *is* the hangup. So the rare accident becomes the ordinary path, and we have to do by hand what the tty would have done for us.

Killing the shell is enough to kill what it was running. It is a session leader with a controlling terminal, so the kernel SIGHUPs the foreground process group when it dies – the runaway `ping` goes with it, exactly as when a real terminal window closes.

`-T` because this one has no terminal and wants none: it is a command we run *at* the container, not a session anybody is sitting in.

Parameters

- **service** (*str*)
- **pidfile** (*str*)

Return type
Command

attach(*service*)

Attach to a running container's main process, stdin included.

The difference from *exec()* is worth being clear about, because the buttons sit next to each other and only one of them is safe to be careless with. `exec` starts a *new* process alongside the app; you can exit it and nothing has happened to the container. `attach` connects you to the process the container already *is* – pid 1, the app itself. It is how you talk to a REPL an app is serving on its stdin, and how you see output that goes nowhere near the logs.

Deliberately plain: no `--sig-proxy`, no `--detach-keys`. Whatever `docker compose attach` does with a signal is docker's business and its documented behaviour, and a person who typed this line into a shell would get exactly what this button gives them. Bolting on flags the user did not ask for, however well meant, is the point at which the UI stops being a face over the command line and starts being a different program.

Parameters
service (*str*)

Return type
Command

validate()

Ask docker compose to parse the file and say nothing if it is valid.

Return type
Command

containers()

Return the stack's containers as reported by `docker compose ps`.

Returns an empty list if docker is unreachable or the project has never been created, because "I cannot tell you about this stack" and "this stack has no containers" look the same to a user, and neither is worth crashing a page render over. A machine with no docker installed should still get a UI that loads and tells them so.

Return type
list[dict]

status()

A one-word summary: `running`, `partial`, `stopped`, or `down`.

The three-way split matters more than it looks. `docker compose up -d` returns once it has *started* the containers, which is a moment before the daemon reports them as `running` – so a status

check immediately after an up can legitimately see `created`. Calling that "stopped" would be a lie, and one a script could act on. Anything that is neither running nor dead is therefore `partial`: in flux, ask again shortly.

Return type
str

running_services()

The services with a container up right now, sorted, for exec and attach.

Running, and not merely declared in the compose file, because both of the things this feeds are things you can only do to a *process*. Offering a person the name of a service that is not up would be offering them a button whose only possible outcome is docker telling them so.

Which also means this is a snapshot and not a promise: a container can exit between the menu being drawn and the user picking from it. The failure is then docker's to report, which it does perfectly well – there is no race worth closing here, only one worth not pretending we have closed.

Return type
list[str]

stats(*container_ids*, *, *as_json=False*)

Live CPU and memory for the given containers, one snapshot then done.

Plain `docker stats` rather than `docker compose stats`, because there is no such compose subcommand – stats is a daemon-level view, and the way to scope it to one project is to name that project's containers. So the caller passes the ids, which come from *containers()*; run with none, `docker stats` would report *every* container on the host, which is the one thing this must not do.

`--no-stream` because a page is not a dashboard: we want the number as it is right now and then our prompt back, not a live feed that never returns.

`as_json` is for the code that has to parse this; the CLI leaves it off and gets docker's own table, which is already the thing a person would want to read.

Parameters

- **container_ids** (*Sequence[str]*)
- **as_json** (*bool*)

Return type
Command

usage()

This stack's CPU and memory use right now, summed over its containers.

`None` when there is nothing to measure – the stack is down, docker is unreachable, or the snapshot could not be read. A stopped app consuming no CPU and no memory is not the same as "0%", and saying nothing is the honest way to draw the difference: the status badge already says it is stopped.

Only *running* containers are asked about. `docker stats` will happily take a stopped container's id and report a row of dashes for it, which parses as nothing and would only slow the snapshot down.

Return type
Usage | None

read()

Return the compose file's raw text.

Return type
str

write(*text*)

Write the compose file, refusing to save something invalid.

Two checks, cheapest first: it must parse as YAML, and docker compose itself must accept it. The second is what actually matters – valid YAML that compose rejects would leave a stack the user cannot start.

The file is only replaced once both checks pass, so a rejected save leaves the previous, working file untouched.

Parameters
text (*str*)

Return type
None

exception spiriconfig_docker.stacks.**StackError**

Bases: Exception

Something is wrong with a stack, or with the request made of it.

class spiriconfig_docker.stacks.**Usage**(*cpu_percent*, *mem_bytes*)

Bases: object

A stack's live resource use, summed across its running containers.

Two numbers, because they answer the two questions a user actually asks of an app – "is it working the processor" and "is it eating the memory" – and disk, the third, is deliberately not here: sizing an app's real footprint means sizing its volumes, which the daemon will not tell us cheaply. That is a separate feature with a separate cost, not a column to bolt on here.

Parameters

- **cpu_percent** (*float*)
- **mem_bytes** (*int*)

cpu_percent: **float**

Summed CPU use, where 100.0 is one core fully busy. Can exceed 100.

mem_bytes: **int**

Summed resident memory in use, in bytes.

spiriconfig_docker.stacks.**create**(*settings*, *name*, *text='services:\n whoami:\n image: traefik/whoami\n ports:\n - "8080:80"\n'*)

Create a new compose project: a directory with a `compose.yaml` in it.

The one place SpiriConfig makes a project directory rather than discovering one – see *`compose_dir`*, which is otherwise a tree we only ever read. It is a deliberate, opt-in exception, and it earns its keep the same way *`Stack.write()`* does: the file is validated with `docker compose config` before we call the project made, so a new stack cannot land unstartable.

The asymmetry with the rest of the module is the point. We *created* this directory, so if compose rejects the file we delete the directory again and the machine is exactly as it was – unlike a hand-made project, cleaning up after ourselves here is ours to do. That is the same bargain the app store's `adopt` strikes in reverse: touch only what you made.

Parameters

- **`settings`** (`DockerSettings`)
- **`name`** (*`str`*)
- **`text`** (*`str`*)

Return type
Stack

`spiriconfig_docker.stacks.`**`discover`**(*settings*)

Find every compose project in the configured directory, sorted by name.

A missing compose directory is not an error: it just means there is nothing to manage yet, which is the state of a fresh machine.

Parameters
`settings` (`DockerSettings`)

Return type
list[Stack]

`spiriconfig_docker.stacks.`**`find_compose_file`**(*directory*)

Return the compose file in `directory`, or None if there is not one.

Parameters
`directory` (*`Path`*)

Return type
Path | None

`spiriconfig_docker.stacks.`**`find_dev_override`**(*directory*)

Return the dev-override compose file in `directory`, or None if there is none.

The optional counterpart to *`find_compose_file()`*: a file dev mode layers on top of the base compose. Its presence alone does not make a directory a stack – *`discover()`* still needs a base file to find one at all.

Parameters
`directory` (*`Path`*)

Return type
Path | None

spiriconfig_docker.stacks.**get**(*settings*, *name*)

Return the named stack, or raise *StackError* if it is unknown.

Names are matched against what is actually on disk, so a caller cannot reach outside the compose directory by passing something like `../../etc`.

Parameters

- **settings** (DockerSettings)
- **name** (*str*)

Return type
Stack

Docker plugin settings, read from the environment.

class spiriconfig_docker.config.**DockerSettings**(*_case_sensitive=None*, *_nested_model_default_partial_update=None*, *_env_prefix=None*, *_env_prefix_target=None*, *_env_file=PosixPath('.')*, *_env_file_encoding=None*, *_env_ignore_empty=None*, *_env_nested_delimiter=None*, *_env_nested_max_split=None*, *_env_parse_none_str=None*, *_env_parse_enums=None*, *_cli_prog_name=None*, *_cli_parse_args=None*, *_cli_settings_source=None*, *_cli_parse_none_str=None*, *_cli_hide_none_type=None*, *_cli_avoid_json=None*, *_cli_enforce_required=None*, *_cli_use_class_docs_for_groups=None*, *_cli_exit_on_error=None*, *_cli_prefix=None*, *_cli_flag_prefix_char=None*, *_cli_implicit_flags=None*, *_cli_ignore_unknown_args=None*, *_cli_kebab_case=None*, *_cli_shortcuts=None*, *_secrets_dir=None*, *_build_sources=None*, ***, *compose_dir=<factory>*, *docker_bin='docker'*, *command_timeout=300.0*)

Bases: BaseSettings

Settings for the docker plugin, prefixed SPIRICONFIG_DOCKER_.

Parameters

- **_case_sensitive** (*bool | None*)
- **_nested_model_default_partial_update** (*bool | None*)
- **_env_prefix** (*str | None*)
- **_env_prefix_target** (*EnvPrefixTarget | None*)
- **_env_file** (*DotenvType | None*)
- **_env_file_encoding** (*str | None*)
- **_env_ignore_empty** (*bool | None*)
- **_env_nested_delimiter** (*str | None*)
- **_env_nested_max_split** (*int | None*)
- **_env_parse_none_str** (*str | None*)
- **_env_parse_enums** (*bool | None*)
- **_cli_prog_name** (*str | None*)
- **_cli_parse_args** (*bool | list[str] | tuple[str, ...] | None*)
- **_cli_settings_source** (*CliSettingsSource[Any] | None*)
- **_cli_parse_none_str** (*str | None*)
- **_cli_hide_none_type** (*bool | None*)
- **_cli_avoid_json** (*bool | None*)
- **_cli_enforce_required** (*bool | None*)
- **_cli_use_class_docs_for_groups** (*bool | None*)
- **_cli_exit_on_error** (*bool | None*)
- **_cli_prefix** (*str | None*)
- **_cli_flag_prefix_char** (*str | None*)
- **_cli_implicit_flags** (*bool | Literal['dual', 'toggle'] | None*)
- **_cli_ignore_unknown_args** (*bool | None*)
- **_cli_kebab_case** (*bool | Literal['all', 'no_enums'] | None*)
- **_cli_shortcuts** (*Mapping[str, str | list[str]] | None*)
- **_secrets_dir** (*PathType | None*)
- **_build_sources** (*tuple[tuple[PydanticBaseSettingsSource, ...], dict[str, Any]] | None*)
- **compose_dir** (*Path*)
- **docker_bin** (*str*)

- **command_timeout** (*float*)

model_config = {'arbitrary_types_allowed': True, 'case_sensitive': False, 'cli_avoid_json': False, 'cli_enforce_required': False, 'cli_exit_on_error': True, 'cli_flag_prefix_char': '-', 'cli_hide_none_type': False, 'cli_ignore_unknown_args': False, 'cli_implicit_flags': False, 'cli_kebab_case': False, 'cli_parse_args': None, 'cli_parse_none_str': None, 'cli_prefix': '', 'cli_prog_name': None, 'cli_shortcuts': None, 'cli_use_class_docs_for_groups': False, 'enable_decoding': True, 'env_file': '.env', 'env_file_encoding': None, 'env_ignore_empty': False, 'env_nested_delimiter': None, 'env_nested_max_split': None, 'env_parse_enums': None, 'env_parse_none_str': None, 'env_prefix': 'SPIRICONFIG_DOCKER_', 'env_prefix_target': 'variable', 'extra': 'ignore', 'json_file': None, 'json_file_encoding': None, 'nested_model_default_partial_update': False, 'protected_namespaces': ('model_validate', 'model_dump', 'settings_customise_sources'), 'secrets_dir': None, 'toml_file': None, 'validate_default': True, 'yaml_config_section': None, 'yaml_file': None, 'yaml_file_encoding': None}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

compose_dir: Path

Directory holding one subdirectory per compose project.

`SPIRICONFIG_DOCKER_COMPOSE_DIR`. We only ever look one level deep, and we never move or delete project directories – the user owns this tree. The one exception is *`create()`*, which makes a new project directory on explicit request and deletes it again if the compose file it just wrote does not validate; it touches only what it made, and nothing that was already here.

Defaults to `/srv/compose` as root and `~/spiri-apps` otherwise – the same answer `spiri-config install` writes, so a bare `spiriconfig serve` and the installed service agree (see `spiriconfig.paths`). A checkout points this at `test_data/compose` through its `.env`, which `./scripts/test-data.sh` writes, so a developer's real apps are left alone.

docker_bin: str

The docker executable. `SPIRICONFIG_DOCKER_DOCKER_BIN`.

command_timeout: float

Seconds before a compose command is considered hung.

`SPIRICONFIG_DOCKER_COMMAND_TIMEOUT`. Only applies to captured commands; streamed ones (`up`, `logs`) are not subject to it.

`spiriconfig_docker.config.`**`docker_settings`**`()`

Load docker plugin settings from the environment.

Return type
DockerSettings

`x-spiri-settings`: the settings form an app declares for itself.

Not to be confused with *spiriconfig_docker.config.DockerSettings*, which is how *SpiriConfig* is configured. This is how an *app* says which of its knobs a human is allowed to turn.

A compose file declares a list of fields under a top-level `x-spiri-settings` key, and each field names a `.env` variable and the NiceGUI widget that should edit it:

```
x-spiri-settings:
  - env: GRAFANA_PORT
    widget: number
    label: HTTP port
    help: The port the dashboard is served on.
    default: "3000"
    min: 1
    max: 65535

  - env: GRAFANA_ADMIN_PASSWORD
    widget: password
    label: Admin password
    required: true

  - env: GRAFANA_LOG_LEVEL
    widget: select
    label: Log level
    options: [debug, info, warn, error]
    default: info
    advanced: true

services:
  grafana:
    image: grafana/grafana:11.1.0
    ports:
      - "${GRAFANA_PORT:-3000}:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: "${GRAFANA_ADMIN_PASSWORD:-admin}"
```

Three things about that are load-bearing.

``x-`` means compose ignores it. Extension fields are part of the compose spec, so a compose file carrying a form is still an ordinary compose file. It runs, unchanged, on a machine that has never heard of SpiriConfig – which is the rule the whole project is built on.

The form writes a ``.env``, and nothing else. Not a database of ours, not a sidecar. `.env` is a file compose already reads (see `spiriconfig_docker.env`), so the settings page is a *nicer way to edit a file the user could have edited in vim*, and every value it sets is one they can read back with `cat`. There is no state here that outlives the file.

The widget is named, not inferred. `widget: select` means `ui.select`. The alternative – declaring an abstract type and having us guess a widget from it – means an app author who wants a dropdown instead of a text box has to discover which incantation of `type:` and `enum:` we happen to map to one. Naming the widget is shorter, and it is honest about what it will do.

`advanced: true` is the app author's way of saying *this knob is for a developer* – a log level, a debug flag, a tuning parameter nobody should have to read past on their way to the port number. The field is then only rendered in advanced mode (see *spiriconfig.advanced*), which makes a long form short for the person who did not want it long. It hides the *widget*, and nothing else: the variable is still read from the `.env`, still written back on save, and the CLI still lists and sets it. An app author choosing to declutter a form is not choosing who may configure their app, and if this ever grew teeth it would be a permission system built out of somebody's UI preference.

The form does not have to live in the compose file. It can sit in a sidecar – `spiri-settings.yaml`, beside it – and then the compose file needs no `x-spiri-settings` key, or indeed any edit at all:

```
grafana/
├── compose.yaml            <- exactly as its author published it
├── spiri-settings.yaml     <- the form
└── .env                    <- the answers
```

That is for the app store's benefit. An app is very often somebody else's compose file, copied in as it was published, and every line a maintainer changes in it is a line they get to re-merge by hand every time upstream moves. A sidecar adds a settings form while touching nothing, so the app stays a clean copy of the thing it came from. See `_source()` for the three forms and how they are chosen between.

The fallback in the compose file (`${GRAFANA_PORT:-3000}`) is not redundant with `default:`. `default:` is what the *form* offers; the compose fallback is what the *app* does when nobody has filled the form in. An app that only had the former would not start until someone had visited a web page, which would be a poor thing to require of a compose file.

`spiriconfig_docker.settings.`**`CHOICE_WIDGETS = frozenset({'radio', 'select', 'toggle'})`**

Widgets that are a choice between fixed options, and so require `options:`.

`spiriconfig_docker.settings.`**`DISCOVERED_WIDGETS = frozenset({'video_device'})`**

Widgets whose options are never only what is declared. `video_device` fills its dropdown with whatever `/dev/video*` exists at render time, adds any URLs an author declares in `options:` on top, and accepts a value typed in by hand besides – so unlike *CHOICE_WIDGETS*, `options:` is optional here, and there is no fixed list to validate a saved value against: a camera present on save may be gone by the time the `.env` is read again, and a typed-in URL was never on any list to begin with.

`spiriconfig_docker.settings.`**`ENV_FILENAME = '.env'`**

The file the form writes, in the stack's own directory. Compose's own default, not a name of ours – which is why an app configured through the UI behaves identically under a bare `docker compose up`.

`spiriconfig_docker.settings.`**`SETTINGS_KEY = 'x-spiri-settings'`**

The compose key an app declares its settings form under.

`spiriconfig_docker.settings.`**`SIDECAR_FILENAMES = ('spiri-settings.yaml', 'spiri-settings.yml')`**

Sidecar names, checked in order, for an app that keeps its settings in their own file rather than in the compose file. See `_source()` for why it can.

`spiriconfig_docker.settings.`**`WIDGETS = frozenset({'checkbox', 'color', 'input', 'number', 'password', 'radio', 'select', 'slider', 'switch', 'textarea', 'toggle', 'video_device'})`**

Every widget an app may ask for, and the NiceGUI element each one becomes.

Kept here rather than in *spiriconfig_docker.widgets* so that the CLI and the schema can be checked without importing NiceGUI. The two are held together by a test that fails if a name here has no builder there.

class spiriconfig_docker.settings.**Field**(*env*, *widget='input'*, *label=''*, *help=''*, *default=''*, *options=<factory>*, *min=None*, *max=None*, *step=None*, *required=False*, *pattern=''*, *advanced=False*)

Bases: object

One setting: a `.env` variable, and how to ask a human for it.

Parameters

- **env** (*str*)
- **widget** (*str*)
- **label** (*str*)
- **help** (*str*)
- **default** (*str*)
- **options** (*list[str]*)
- **min** (*float | None*)
- **max** (*float | None*)
- **step** (*float | None*)
- **required** (*bool*)
- **pattern** (*str*)
- **advanced** (*bool*)

env: str

The `.env` variable this field reads and writes.

label: str

Shown beside the widget. Defaults to a readable form of *env*.

help: str

Shown under the widget. This is where an app author explains themselves.

default: str

What the form offers when the `.env` does not set this variable.

options: list[str]

The choices, for *CHOICE_WIDGETS*.

pattern: str

A regular expression the value must match. Checked in the form, and on save.

advanced: bool

Show this field only in advanced mode. Clutter, not secrecy.

A hidden field is still filled in from the `.env`, still written back when the form is saved, and still `spiriconfig docker settings`'s business. See the module docstring, and *spiriconfig.advanced*, which says the same thing about every other advanced-only element on the page.

property title: str

what the author wrote, or a readable fallback.

`GRAFANA_ADMIN_PASSWORD` becomes `Grafana admin password`, so an author who cannot be bothered to write a label still gets something better than a shouting environment variable.

Type
The label to render

exception spiriconfig_docker.settings.**SettingsError**

Bases: `Exception`

An app's `x-spiri-settings` is malformed, or a value for it is.

class spiriconfig_docker.settings.**StackSettings**(*stack*, *fields*)

Bases: `object`

A stack's declared form, bound to the `.env` it reads and writes.

Parameters

- **stack** (`Stack`)
- **fields** (*list[Field]*)

values()

What the form should show: the `.env`'s value, or the field's default.

Only declared fields. Whatever else the user keeps in their `.env` is theirs, and none of the form's business – it is not shown, and *save()* will not touch it.

Return type
dict[str, str]

read()

The `.env`'s text, or `""` if the app has not got one yet.

A missing `.env` is the ordinary state of an app nobody has configured, not an error – and an empty buffer is the honest thing to show for one.

Return type
str

preview(*values*)

The exact text *save()* would write. Shown in the UI before it does.

The same instinct as showing the command line before running it: a settings page that edits a file should be willing to say which bytes it is about to put in it. *Exact* is meant literally – the advanced editor is filled with this text and will write back whatever it then holds, so a preview that differed

from the write by so much as a header would be a preview that lied the moment somebody edited it.

Parameters
values (*dict[str, str]*)

Return type
str

save(*values*)

Write the values to the `.env`, refusing to leave a broken one behind.

The keys the form declares are patched into the file; everything else in it – the user's comments, their own variables, the order they chose – comes through untouched. See `spiriconfig_docker.env.patch()`.

Parameters
values (*dict[str, str]*)

Return type
None

write(*text*)

Write `text` to the `.env` as given, refusing to leave a broken one.

The other door into the same file, for the advanced editor: bytes in, bytes on disk. No patching, no header, no schema – a file somebody edited by hand is a file they own, and the form's opinion about which keys exist is not one it gets to impose on a text editor. What they typed is what compose will read, which is the entire reason to offer a text editor at all.

The check is the same bargain *spiriconfig_docker.stacks.Stack.write()* makes for the compose file, and for the same reason: compose has to be able to read this file, and a `.env` it rejects makes the stack unstartable – including by hand, from a shell, which is the escape hatch that must never close. So the new file goes down, `docker compose config` is asked whether it can still read the project, and the original is put back if it says no. A rejected save leaves the previous, working file exactly as it was.

Parameters
text (*str*)

Return type
None

spiriconfig_docker.settings.**declared**(*compose_file*)

The fields an app declares, or `[]` if it declares none.

Reading the compose file as YAML is fine, and is what *spiriconfig_appstore.stores.App.version()* already does. We never write it back – the file the form edits is the `.env` beside it, precisely so that a settings page can never reformat somebody's compose file or eat its comments.

Raises *SettingsError* if the declaration is there but wrong, because a store author who typoed `widgit:` needs to hear about it. An app with no settings at all is not an error; it is simply an app with no settings, which is most of them.

Parameters
compose_file (*Path*)

Return type
list[Field]

spiriconfig_docker.settings.**find_sidecar**(*directory*)

The settings sidecar in `directory`, or None if there is not one.

The same shape as *spiriconfig_docker.stacks.find_compose_file()*, and for the same reason: a fixed list of names, checked in order, so that finding one is a question about the filesystem and not about configuration.

Parameters
directory (*Path*)

Return type
Path | None

spiriconfig_docker.settings.**for_stack**(*stack*)

The settings form for `stack`, empty if it declares none.

Raises *SettingsError* if it declares a malformed one.

Parameters
stack (Stack)

Return type
StackSettings

spiriconfig_docker.settings.**get**(*stack_settings*, *key*)

Look up a declared field by its variable name.

Parameters

- **stack_settings** (StackSettings)
- **key** (*str*)

Return type
Field

spiriconfig_docker.settings.**has_settings**(*stack*)

Whether `stack` declares a settings form that we could render.

Swallows a malformed declaration, deliberately: this is called while drawing a list of every stack, and one app with a typo in it must not take the page down with it. The error is logged, and it is raised properly by *for_stack()* when someone actually opens that app's settings – which is the moment they can do something about it.

Parameters
stack (Stack)

Return type
bool

The widgets a settings form is made of, one per NiceGUI element.

`widget: select` in a compose file means `ui.select` on the page. That is the whole mapping, and keeping it that direct is a deliberate refusal of the usual design, in which a schema declares an abstract *type* and the renderer guesses a widget from it. Guessing is what forces an app author to work out that `type: string` plus `enum:` is the incantation for a dropdown, and it is what leaves them stuck when they want a slider instead of a number box for the same integer. Naming the element is shorter to write, obvious to read, and puts the author in charge of their own form.

Every widget has to do three things, and they are separate because a `.env` holds text and a widget does not:

- **build** the NiceGUI element
- **parse** the text out of the `.env` into whatever the element's `value` is
- **format** that value back into text for the `.env`

The round trip is where the bodies are buried. `ui.number` hands back a float, so a port typed as `3000` would go into the file as `3000.0` and into the container as a string docker cannot bind. `ui.switch` hands back a bool, and `str(True)` is `"True"`, which is not how anything else in a container spells it. So each widget owns its own conversions, and *`values()`* is the only thing that reads them back.

spiriconfig_docker.widgets.**MARKER_PREFIX = 'setting-'**

NiceGUI marker put on each rendered widget, followed by the variable it sets: `setting-GRAFANA_PORT`. See *`render()`*.

spiriconfig_docker.widgets.**REGISTRY: dict[str, *Widget*] = {'checkbox': Widget(build=<function _checkbox>, parse=<function _to_bool>, format=<function _from_bool>), 'color': Widget(build=<function _color>, parse=<function _to_text>, format=<function _from_text>), 'input': Widget(build=<function _input>, parse=<function _to_text>, format=<function _from_text>), 'number': Widget(build=<function _number>, parse=<function _to_number>, format=<function _from_number>), 'password': Widget(build=<function _password>, parse=<function _to_text>, format=<function _from_text>), 'radio': Widget(build=<function _radio>, parse=<function _to_text>, format=<function _from_text>), 'select': Widget(build=<function _select>, parse=<function _to_text>, format=<function _from_text>), 'slider': Widget(build=<function _slider>, parse=<function _to_number>, format=<function _from_number>), 'switch': Widget(build=<function _switch>, parse=<function _to_bool>, format=<function _from_bool>), 'textarea': Widget(build=<function _textarea>, parse=<function _to_text>, format=<function _from_text>), 'toggle': Widget(build=<function _toggle>, parse=<function _to_text>, format=<function _from_text>), 'video_device': Widget(build=<function _video_device>, parse=<function _to_text>, format=<function _from_text>)}**

Every widget name an app may ask for. The keys are exactly *`spiriconfig_docker.settings.WIDGETS`* – a test asserts it, so a widget added to one and forgotten in the other fails the suite rather than the page.

spiriconfig_docker.widgets.**RESET_PREFIX = 'reset-'**

NiceGUI marker put on each field's reset button, followed by the variable it resets: `reset-GRAFANA_PORT`. See *`render()`*.

spiriconfig_docker.widgets.**TRUTHY = frozenset({'1', 'on', 'true', 'yes'})**

Text a `.env` might hold for a boolean widget. Anything else is false.

Generous on the way in and strict on the way out: we write `true`/`false`, but a user who hand-edited their `.env` to say `yes` meant yes, and a switch that silently showed their `yes` as off would be lying to them.

class spiriconfig_docker.widgets.**Bound**(*field*, *element*, *widget*)

Bases: `object`

A rendered field, and the element holding its answer.

Parameters

- **field** (`Field`)
- **element** (*Any*)
- **widget** (`Widget`)

property value: str

What the user has put in, as the `.env` will hold it.

property default: Any

The field's declared default, parsed into the element's `value` type.

The same trip the `.env`'s text takes on the way into the widget, so that comparing it against `element.value` is comparing like with like – a number widget holds `8080.0`, and `field.default` is the string `"8080"`, and only one of those is what the widget will read back.

reset()

Put the widget back to the field's declared default.

Only the widget – nothing is written. The reset is undone by a Cancel and made real by a Save, exactly like typing into the box by hand, because that is all it is: *`value()`* reads the element back whatever put the value there.

Return type
None

class spiriconfig_docker.widgets.**Widget**(*build*, *parse*, *format*)

Bases: `object`

How one `widget:` name is built, and how its value crosses the `.env`.

Parameters

- **build** (*Callable[[Field, Any], Any]*)
- **parse** (*Callable[[str, Field], Any]*)
- **format** (*Callable[[Any], str]*)

parse: Callable[[str, *Field*], Any]

Text from the `.env` -> the element's `value`.

format: `Callable[[Any], str]`

The element's `value` -> text for the `.env`.

`spiriconfig_docker.widgets.`**form**(*fields*, *values*)

Render the whole form, in the order the app declared it.

Parameters

- **fields** (*list[Field]*)
- **values** (*dict[str, str]*)

Return type
list[Bound]

`spiriconfig_docker.widgets.`**render**(*field*, *value*)

Render one field, showing `value`, and return a handle on it.

An `advanced:` field is rendered like any other and then marked, so that the switch shows and hides it live – and, crucially, so that it is *built* either way. The element exists whatever the switch says; *values()* reads it back whatever the switch says. Hiding a widget must not drop the variable out of the `.env`, which is what skipping the build would quietly do to anyone who saved a form with advanced mode off.

The whole column is marked rather than the widget alone. A field is a widget *and* its help text, and half a field left on the page would be worse than none.

The widget stays a *direct* child of the column, with the help text and the reset button on a row of their own beneath it. That is not just tidy: the mark lands one level above the widget, and *spiriconfig.advanced.mark()* is on the column, so "the field" the switch reveals and "the field" a test reaches for from the widget are the same element either way.

The reset button appears only when the value has drifted from the default (see *Bound.default()*), and puts the widget back to it. A reset button on a field already at its default is a button that does nothing, and one on every field is clutter, so it shows itself when there is something to undo and hides again the moment the value is back.

Parameters

- **field** (*Field*)
- **value** (*str*)

Return type
Bound

`spiriconfig_docker.widgets.`**values**(*bound*)

Read every rendered field back, as the text a `.env` will hold.

Parameters
bound (*list[Bound]*)

Return type
dict[str, str]

2.9.8 The app store plugin

App stores, and the git commands that act on them.

An *app store* is a git repository with one top-level directory per app, each containing a compose file. An *app* is one of those directories. There is no manifest, no index, and no registry – the repository layout is the format, so a store can be read with `ls` and published with `git push`.

Installing an app **symlinks** it into the compose directory:

```
/srv/compose/whoami -> /var/lib/spiriconfig/stores/spiri-apps/whoami
```

That single decision is what makes the rest of this module small, because it means SpiriConfig records nothing about the install. The symlink *is* the record:

- Where did this app come from? – `readlink`
- Have I changed it? – `git status`
- What did I change? – `git diff`
- What did the store change? – `git diff HEAD @{upstream}`
- Update it – `git merge`
- Uninstall it – `rm` the symlink, which deletes nothing real

Every one of those is a git question with a git answer, on a working tree the user can `cd` into. We are not tracking installs; git is, and it was going to do a better job of it than we would.

As everywhere else in SpiriConfig, functions that act on a store *return* a `Command` rather than running it, so the UI can show the user the git line before it runs and tests can assert on it without a network.

class spiriconfig_appstore.stores.**App**(*store*, *name*, *path*, *compose_file*)

Bases: object

One app in a store: a directory with a compose file in it.

Parameters

- **store** (Store)
- **name** (*str*)
- **path** (*Path*)
- **compose_file** (*Path*)

name: str

The directory name in the store, and the default name once installed.

path: Path

The app's directory inside the store's checkout.

version()

A human-readable label for what is currently checked out.

Two sources, in order: the store maintainer's *VERSION_KEY* if they set one, otherwise the date of the last commit to touch this app.

This is *only* a label. Nothing decides whether to update based on it, because git can answer that question exactly – see `Store.apps_with_updates()`. Which is the escape from the usual trap: we never have to define what "version 2" of a set of containers means, or trust a maintainer to bump a number they forgot about.

Return type
str

log_command()

`git log` for the last commit touching this app. `%cs` is a date.

Return type
Command

local_diff()

What the user has changed in this app, against the store's version.

Return type
Command

upstream_diff()

What the store has changed in this app since we last pulled.

Return type
Command

changed_locally()

The app's *tracked* files that the user has edited.

Tracked files only, and that is deliberate. A compose file with a relative bind mount (`./config:/config`) makes docker create data directories inside the store's checkout, which show up as untracked files. Those are the user's data, not an edit to the app, and counting them as a modification would tell every user their apps were "modified" the moment they started one.

Return type
list[str]

changed_upstream()

The app's files the store has changed since the version we have.

Return type
list[str]

link_path(*compose_dir*, *name=None*)

Where this app's symlink would live in the compose directory.

Parameters

- **compose_dir** (*Path*)
- **name** (*str | None*)

Return type
Path

install_command(*compose_dir*, *name=None*)

The `ln -s` that installs this app.

Rendered as a command rather than done with `os.symlink()` for the usual reason: it is the line the user could have typed, and `--show` has to be able to print it.

Parameters

- **compose_dir** (*Path*)
- **name** (*str | None*)

Return type
Command

class spiriconfig_appstore.stores.**Store**(*slug*, *url*, *path*, *settings*)

Bases: `object`

One cloned app store.

Parameters

- **slug** (*str*)
- **url** (*str*)
- **path** (*Path*)
- **settings** (AppStoreSettings)

path: Path

The clone's working tree.

clone_command()

Clone the store.

No `cwd`: both the source and the target are absolute (see *stores()*), so this runs correctly from anywhere, and git creates the leading directories itself.

`credential.helper=store` lets a private store authenticate from the credential a user saved when adding it (see `spiriconfig_appstore.credentials`). It is on the line unconditionally because it is harmless without one – with no matching entry the helper finds nothing and the clone proceeds exactly as before, anonymous for a public store and prompting nowhere – and putting it here keeps the token itself off the command line, where it would otherwise have to live.

Return type
Command

fetch_command()

Ask the remote what it has, without touching the working tree.

Carries `credential.helper=store` for the same reason *clone_command()* does: a private store's periodic `check` has to authenticate too, from the same saved credential, and the flag is inert when there is none.

Return type
Command

merge_command()

Merge the fetched upstream into the checkout.

A merge, not a rebase or a reset: the user's edits to installed apps are commits on this branch (see *commit_local_command()*), and a merge is the operation that reconciles two lines of change while keeping both. If it conflicts, it conflicts in the file, with the markers git always uses, and the user resolves it in the editor they already have.

Carries an identity like `_commit()` does: once both sides have moved, the merge makes a merge commit, and without one git refuses.

Return type
Command

commit_local_command(*message*)

Commit whatever the user has edited, before merging over it.

This is the step that makes an update non-destructive. git will refuse to merge across uncommitted changes, and the tempting fixes – stash, reset, checkout – all end with the user's work somewhere they will not think to look. Committing it puts their edit on the branch, so the merge has to *reconcile* with it, and the worst case is a conflict they can see rather than an edit that silently vanished.

`--all` stages tracked modifications only, which is what we want: the data directories docker creates inside the checkout are untracked, and committing a user's media library into the app store would be a memorable way to fail.

Parameters
message (*str*)

Return type
Command

discard_local_command()

Throw away every local edit in this store. Asked for, never assumed.

Return type
Command

remove_command()

Delete this store's checkout: `rm -rf` the clone.

The counterpart to *clone_command()*, and safe to offer for exactly the reason `adopt` is: we made this directory with `git clone`, so removing it puts the machine back where it was before we added the store. Nothing else here deletes a directory – an app uninstall only unlinks – and this one is allowed to precisely because it is undoing our own `clone` and touching nothing the user created.

Apps installed from the store become dangling symlinks, which the docker plugin and *spiriconfig_appstore.installs.installed()* already tolerate. The caller is expected to warn about them first; git will not, because as far as it is concerned this is just a directory.

Return type
Command

resolve_plan()

Conclude a conflicted merge: stage the fixed files, commit the merge.

Guarded by *unresolved()*, and that guard is not decoration. It is tempting to write this as `git commit --all`, which reads like "commit whatever is there" – and it is a trap. `-a` stages *unmerged* files too, which git treats as "the user has resolved these", so a file still full of `<<<<<<<` markers gets committed into the store as though it were a considered decision. The user gets told the update finished, and the broken file is now the app's history.

So: refuse while any marker remains, and stage explicitly rather than with `-a`. Git will not check this for us – it has no idea what a conflict marker means once the index says the file is resolved.

Return type
list[Command]

unresolved()

Conflicted files that still contain conflict markers.

A text search, because a conflict marker is a text fact. We are not parsing the YAML – a half-merged compose file is usually not valid YAML at all, which is the point.

Return type
list[str]

abort_command()

Undo the merge entirely, back to how things were before the update.

The user's own edits survive this: they were committed *before* the merge began, precisely so that aborting could be a safe thing to offer.

Return type
Command

property in_merge: bool

Whether a merge is in progress and waiting on the user.

conflicts()

Files with unresolved conflict markers in them.

Return type
list[str]

is_dirty()

Whether any tracked file in the store has been edited.

Return type
bool

apps()

Every app in the store: top-level directories with a compose file.

The same rule *spiriconfig_docker.stacks* uses to find a stack, and that is not a coincidence – a store is just a compose directory that happens to live in git. Which is why `git clone` + `cp -r` is a perfectly good way to use one without SpiriConfig at all.

Return type
list[App]

app(*name*)

Return the named app, or raise *StoreError*.

Matched against what is on disk, so a caller cannot escape the store by asking for something like `../../etc`.

Parameters
name (*str*)

Return type
App

exception spiriconfig_appstore.stores.**StoreError**

Bases: Exception

Something is wrong with a store, or with the request made of it.

spiriconfig_appstore.stores.**VERSION_KEY = 'x-spiri-config-version'**

The compose key a store maintainer can set to name a version themselves.

Versioning a bag of containers is not a thing anyone can do well, so we do not ask them to. This is optional, and when it is absent we fall back to the date of the last commit that touched the app – which is always available, always monotonic, and requires no discipline from the maintainer.

Nothing depends on it. See *App.version()*.

spiriconfig_appstore.stores.**check_plan**(*settings*)

A fetch for every cloned store: "check for updates", across the board.

One `git fetch` per store that is actually on disk. A store that is only a seed (not cloned yet) has nothing to check and is skipped – its own Clone action is what brings it down.

This is the whole of what "check for updates" does, and it is deliberately only half of git. Fetching never touches a working tree: it updates git's record of where the remote is, which is exactly what makes the "update available" markers truthful (they compare the checked-out commit against that record – see `App.has_update()`). Nothing installed changes until someone runs an *update_plan()*. That is what makes this safe to run on boot, and safe to run as often as you like.

Parameters
settings (AppStoreSettings)

Return type
list[Command]

spiriconfig_appstore.stores.**find_app**(*settings*, *name*)

Find an app by `name` or `store/name` across every cloned store.

Parameters

- **settings** (AppStoreSettings)
- **name** (*str*)

Return type
App

spiriconfig_appstore.stores.**get_store**(*settings*, *slug*)

Return the named store, or raise *StoreError*.

Parameters

- **settings** (AppStoreSettings)
- **slug** (*str*)

Return type
Store

spiriconfig_appstore.stores.**remote_url**(*settings*, *path*)

Read a checkout's origin URL, or None if it has none we can read.

This is what makes disk the source of truth: a store is a clone, and a clone remembers where it came from, so we ask *it* rather than keeping a list of our own. `git remote get-url origin` is the same question `readlink` answers for an installed app – "where did this come from?" asked of the filesystem.

None for a directory with no `origin` (someone's local-only repo, say): still a perfectly good store to install from, just one with no upstream to show or pull. The caller decides what to display for it.

Parameters

- **settings** (AppStoreSettings)
- **path** (*Path*)

Return type
str | None

spiriconfig_appstore.stores.**slug_for**(*url*)

Derive a directory name from a git URL.

`https://github.com/spiri/spiri-apps.git` -> `spiri-apps`. Purely cosmetic – it names the clone directory and the store in the UI.

Parameters
url (*str*)

Return type
str

spiriconfig_appstore.stores.**store_for_url**(*settings*, *url*)

Build the *Store* that adding `url` would create.

Not necessarily one of *stores()* – this is how a *new* store is named and placed before it exists, so the caller can run its *Store.clone_command()*. Once cloned, *stores()* rediscovers it from disk like any other.

Parameters

- **settings** (AppStoreSettings)
- **url** (*str*)

Return type
Store

spiriconfig_appstore.stores.**stores**(*settings*)

Every store: the clones on disk, plus any not-yet-cloned seeds.

Disk is the source of truth. A store *is* a git checkout under *store_dir*, so the live list is whatever is actually cloned there – which is what lets the UI add one (clone it) and remove one (delete it) without SpiriConfig keeping a list of its own. Each clone's URL is read back off the clone itself; see *remote_url()*.

stores is a *seed* list, not the authority. Any seed whose slug is not already cloned is included as a not-yet-cloned store, so `check` (or the Clone button) can bring it down – and the moment it is cloned, disk discovery takes over and the seed entry is deduplicated away. A store the user cloned themselves and never listed is a first-class store; a seed the user removed stays gone until they clone it again.

Paths are resolved here, once, so that nothing downstream has to remember to.

Parameters
settings (AppStoreSettings)

Return type
list[Store]

spiriconfig_appstore.stores.**update_plan**(*store*, *, *discard_local=False*)

The commands that bring `store` up to date, in order.

A list rather than one command, because an update is genuinely three steps and the middle one is conditional – and the user should watch all three go past rather than have us hide the interesting one.

Note what this does *not* do: update a single app. The symlinks all point into one working tree, which is at one commit, so "update whoami but not nextcloud" is not a state this repository can be in. That sounds like a limitation and is closer to a feature: pulling the store only rewrites files on disk. Nothing is restarted, nothing is pulled from a registry, and no running container changes until someone runs `docker compose up` on that stack. Per-app control still exists – it just lives at up-time, where the user was going to make that decision anyway.

Raises if a previous update is still waiting on a conflict. Merging on top of an unfinished merge is how a confusing situation becomes an unrecoverable one.

Parameters

- **store** (Store)
- **discard_local** (*bool*)

Return type
list[Command]

Installing an app: making, reading, and removing a symlink.

An installed app is a symlink in the compose directory pointing at an app directory inside a store's checkout:

```
/srv/compose/whoami -> /var/lib/spiriconfig/stores/spiri-apps/whoami
```

Which means the docker plugin needs to know nothing about any of this. It walks the compose directory looking for directories with compose files in them, a symlink to a directory *is* a directory, and so an installed app is simply a stack. `spiriconfig docker up whoami` works on it, `cd /srv/compose/whoami && docker compose up -d` works on it, and neither has any idea a store exists.

The symlink is also the only thing we create, which is what keeps the promise in *Design notes* that we do not create or delete the user's project directories. An uninstall removes a symlink; the app, and anything the user put beside it, is still there.

class spiriconfig_appstore.installs.**Install**(*name*, *link*, *target*, *store*, *app_name*)

Bases: object

An app installed into the compose directory, as seen from the symlink.

Parameters

- **name** (*str*)
- **link** (*Path*)
- **target** (*Path*)
- **store** (Store)
- **app_name** (*str*)

name: str

The symlink's name, which is the compose project name.

link: Path

The symlink itself, in the compose directory.

target: Path

an app directory inside a store checkout.

Type
Where it points

app_name: str

Which app in which store. Read back off the symlink; we store it nowhere.

uninstall_command()

Remove the symlink.

`rm`, with no `-r` and no `-f`: this deletes a link and cannot delete a directory, so the worst a bug here can do is unlink something. The app's files stay in the store, and so does anything the user left in it – their data outlives an uninstall, which is the behaviour they will assume anyway and the one they cannot recover from if we get it wrong.

Return type
Command

adopt_commands()

Replace the symlink with a real copy of the app's directory.

The escape hatch, and the reason nobody is trapped by the symlink design. An adopted app is an ordinary compose project: it never updates again, it has no store, and the user owns it completely. It is what they would have had if they had run `cp -r` themselves, which is exactly what this runs.

Return type
list[Command]

spiriconfig_appstore.installs.**install_command**(*app*, *compose_dir*, *name=None*)

Build the `ln -s` that installs `app`, refusing to clobber anything.

Checked before the command is built rather than letting `ln` fail, because the failure we most want to prevent is the one where the target is the user's own hand-made project of the same name. `ln` would refuse anyway, but it would refuse with "File exists", and the user deserves to be told which of the several quite different things went wrong.

Parameters

- **app** (App)
- **compose_dir** (*Path*)
- **name** (*str | None*)

Return type
Command

spiriconfig_appstore.installs.**installed**(*settings*, *compose_dir*)

Every store-installed app in the compose directory, sorted by name.

Parameters

- **settings** (AppStoreSettings)
- **compose_dir** (*Path*)

Return type
list[Install]

spiriconfig_appstore.installs.**uninstall**(*settings*, *compose_dir*, *name*)

Look up an installed app by name, or raise `StoreError`.

Refuses anything that is not one of our symlinks. A user who made `/srv/compose/whoami` themselves, or adopted it, gets told to remove it themselves – we did not create that directory, so we will not delete it.

Parameters

- **settings** (AppStoreSettings)
- **compose_dir** (*Path*)
- **name** (*str*)

Return type
Install

App store settings, read from the environment.

class spiriconfig_appstore.config.**AppStoreSettings**(*_case_sensitive=None*, *_nested_model_default_partial_update=None*, *_env_prefix=None*, *_env_prefix_target=None*, *_env_file=PosixPath('.')*, *_env_file_encoding=None*, *_env_ignore_empty=None*, *_env_nested_delimiter=None*, *_env_nested_max_split=None*, *_env_parse_none_str=None*, *_env_parse_enums=None*, *_cli_prog_name=None*, *_cli_parse_args=None*, *_cli_settings_source=None*, *_cli_parse_none_str=None*, *_cli_hide_none_type=None*, *_cli_avoid_json=None*, *_cli_enforce_required=None*, *_cli_use_class_docs_for_groups=None*, *_cli_exit_on_error=None*, *_cli_prefix=None*, *_cli_flag_prefix_char=None*, *_cli_implicit_flags=None*, *_cli_ignore_unknown_args=None*, *_cli_kebab_case=None*, *_cli_shortcuts=None*, *_secrets_dir=None*, *_build_sources=None*, ***, *stores=[]*, *store_dir=<factory>*, *git_bin='git'*, *command_timeout=300.0*, *commit_name='SpiriConfig'*, *commit_email='spiriconfig@localhost'*)

Bases: BaseSettings

Settings for the appstore plugin, prefixed SPIRICONFIG_APPSTORE_.

Parameters

- **_case_sensitive** (*bool | None*)
- **_nested_model_default_partial_update** (*bool | None*)
- **_env_prefix** (*str | None*)
- **_env_prefix_target** (*EnvPrefixTarget | None*)
- **_env_file** (*DotenvType | None*)

- **_env_file_encoding** (*str | None*)
- **_env_ignore_empty** (*bool | None*)
- **_env_nested_delimiter** (*str | None*)
- **_env_nested_max_split** (*int | None*)
- **_env_parse_none_str** (*str | None*)
- **_env_parse_enums** (*bool | None*)
- **_cli_prog_name** (*str | None*)
- **_cli_parse_args** (*bool | list[str] | tuple[str, ...] | None*)
- **_cli_settings_source** (*CliSettingsSource[Any] | None*)
- **_cli_parse_none_str** (*str | None*)
- **_cli_hide_none_type** (*bool | None*)
- **_cli_avoid_json** (*bool | None*)
- **_cli_enforce_required** (*bool | None*)
- **_cli_use_class_docs_for_groups** (*bool | None*)
- **_cli_exit_on_error** (*bool | None*)
- **_cli_prefix** (*str | None*)
- **_cli_flag_prefix_char** (*str | None*)
- **_cli_implicit_flags** (*bool | Literal['dual', 'toggle'] | None*)
- **_cli_ignore_unknown_args** (*bool | None*)
- **_cli_kebab_case** (*bool | Literal['all', 'no_enums'] | None*)
- **_cli_shortcuts** (*Mapping[str, str | list[str]] | None*)
- **_secrets_dir** (*PathType | None*)
- **_build_sources** (*tuple[tuple[PydanticBaseSettingsSource, ...], dict[str, Any]] | None*)
- **stores** (*list[str]*)
- **store_dir** (*Path*)
- **git_bin** (*str*)
- **command_timeout** (*float*)
- **commit_name** (*str*)
- **commit_email** (*str*)

model_config = {'arbitrary_types_allowed': True, 'case_sensitive': False, 'cli_avoid_json': False, 'cli_enforce_required': False, 'cli_exit_on_error': True, 'cli_flag_prefix_char': '-', 'cli_hide_none_type': False, 'cli_ignore_unknown_args': False, 'cli_implicit_flags': False, 'cli_kebab_case': False, 'cli_parse_args': None, 'cli_parse_none_str': None, 'cli_prefix': '', 'cli_prog_name': None, 'cli_shortcuts': None, 'cli_use_class_docs_for_groups': False, 'enable_decoding': True, 'env_file': '.env', 'env_file_encoding': None, 'env_ignore_empty': False, 'env_nested_delimiter': None, 'env_nested_max_split': None, 'env_parse_enums': None, 'env_parse_none_str': None, 'env_prefix': 'SPIRICONFIG_APPSTORE_', 'env_prefix_target': 'variable', 'extra': 'ignore', 'json_file': None, 'json_file_encoding': None, 'nested_model_default_partial_update': False, 'protected_namespaces': ('model_validate', 'model_dump', 'settings_customise_sources'), 'secrets_dir': None, 'toml_file': None, 'validate_default': True, 'yaml_config_section': None, 'yaml_file': None, 'yaml_file_encoding': None}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

stores: list[str]

Seed git URLs of app stores. `SPIRICONFIG_APPSTORE_STORES`, a JSON list.

An app store is an ordinary git repository with one top-level directory per app, each containing a compose file. Nothing else. That is the whole format, which means anyone can host one, and a user can inspect one with `git clone` and read it with `ls`.

Not the live list – disk is (see *spiriconfig_appstore.stores.stores()*). These are stores to *offer* on a machine that has none yet: each one that is not already cloned shows up ready to clone, and once cloned it is an ordinary store discovered from disk like any the user added themselves. Removing a store in the UI deletes its checkout; a seed listed here then reappears as not-yet-cloned, because this list says what to offer, not what must exist.

Empty by default: there is no store every machine should be offered. A checkout seeds the example store this repository ships, through the `.env ./scripts/test-data.sh` writes. A git URL and a local path are the same thing to `git clone`, which is why the example store is a perfectly ordinary store and not a special case anywhere in the code.

store_dir: Path

Where store clones live. `SPIRICONFIG_APPSTORE_STORE_DIR`.

Not a cache: installed apps are symlinks *into* these clones, and a user's edits to an installed app land here as changes in a git working tree. That is the point – it is what lets `git diff` answer "what did I change?" and `git merge` answer "what happens when the store moves on?".

Defaults to `/var/lib/spiriconfig/stores` as root and `~/.local/share/spiriconfig/stores` otherwise (see `spiriconfig.paths`); a checkout points it at `test_data/stores` through its `.env`.

git_bin: str

The git executable. `SPIRICONFIG_APPSTORE_GIT_BIN`.

command_timeout: *float*

Seconds before a git command is considered hung.

commit_email: *str*

Identity for the local commits made to preserve a user's edits on update.

git refuses to commit without one, and the machine running SpiriConfig usually has no global git identity configured. Passed with `-c` on the command line rather than written into the repo's config, so the commands we print stay complete and copy-pasteable.

spiriconfig_appstore.config.**appstore_settings**()

Load appstore plugin settings from the environment.

Return type
AppStoreSettings

2.10 Revision history

This table is generated from `revisions.yaml` at the root of this repository. Do not edit it here – add an entry there and it appears below, on the cover page, and in the PDF footer.

Revision	Date	Summary	Author
0.1 (draft)	2026-01-01	Initial draft.	Alex Davies

2.10.1 Issuing a revision

The document's revision is separate from the software's version: revise the document when what it says changes.

1. Add an entry to the bottom of `revisions.yaml`, without `draft: true`.
2. Merge it. The next software release (`vX.Y.Z` tag) attaches the PDF of this revision to its release. That release asset is the controlled copy: link people to it rather than to a PDF built from a branch, which can change under them.

PYTHON MODULE INDEX

S

spiriconfig.advanced, 38
spiriconfig.commands, 42
spiriconfig.config, 48
spiriconfig.plugins, 36
spiriconfig.preferences, 40
spiriconfig.terminal, 46
spiriconfig_appstore.config, 83
spiriconfig_appstore.installs, 80
spiriconfig_appstore.stores, 73
spiriconfig_docker.config, 61
spiriconfig_docker.settings, 63
spiriconfig_docker.stacks, 53
spiriconfig_docker.widgets, 70

INDEX

A

abort_command() (*spiriconfig_appstore.stores.Store method*), 77
adopt_commands() (*spiriconfig_appstore.installs.Install method*), 81
advanced (*spiriconfig_docker.settings.Field attribute*), 66
advanced (*spiriconfig.config.Settings attribute*), 51
advanced (*spiriconfig.plugins.Plugin attribute*), 37
AdvancedState (*class in spiriconfig.advanced*), 39
App (*class in spiriconfig_appstore.stores*), 73
app() (*spiriconfig_appstore.stores.Store method*), 78
app_name (*spiriconfig_appstore.installs.Install attribute*), 81
apps() (*spiriconfig_appstore.stores.Store method*), 77
appstore_settings() (*in module spiriconfig_appstore.config*), 86
AppStoreSettings (*class in spiriconfig_appstore.config*), 83
attach() (*spiriconfig_docker.stacks.Stack method*), 57
auth (*spiriconfig.config.Settings attribute*), 52
auth_group (*spiriconfig.config.Settings attribute*), 53
auth_service (*spiriconfig.config.Settings attribute*), 52

B

Bound (*class in spiriconfig_docker.widgets*), 71
BROWSER_STORAGE_KEY (*in module spiriconfig.advanced*), 39
BrowserPreferences (*class in spiriconfig.preferences*), 41

C

changed_locally() (*spiriconfig_appstore.stores.App method*), 74
changed_upstream() (*spiriconfig_appstore.stores.App method*), 74
check() (*spiriconfig.commands.Result method*), 43
check_plan() (*in module spiriconfig_appstore.stores*), 78
CHOICE_WIDGETS (*in module spiriconfig_docker.settings*), 65
cli() (*spiriconfig.plugins.Plugin method*), 37
clone_command() (*spiriconfig_appstore.stores.Store method*), 75
close() (*spiriconfig.commands.PtySession method*), 45
Command (*class in spiriconfig.commands*), 42
command_timeout (*spiriconfig_appstore.config.AppStoreSettings attribute*), 86
command_timeout (*spiriconfig_docker.config.DockerSettings attribute*), 63
CommandError, 43
commit_email (*spiriconfig_appstore.config.AppStoreSettings attribute*), 86
commit_local_command() (*spiriconfig_appstore.stores.Store method*), 76
compose_dir (*spiriconfig_docker.config.DockerSettings attribute*), 63
compose_file (*spiriconfig_docker.stacks.Stack attribute*), 54
COMPOSE_FILENAMES (*in module spiriconfig_docker.stacks*), 53
conflicts() (*spiriconfig_appstore.stores.Store method*), 77
containers() (*spiriconfig_docker.stacks.Stack method*), 57
cpu_percent (*spiriconfig_docker.stacks.Usage attribute*), 59
create() (*in module spiriconfig_docker.stacks*), 59

D

declared() (*in module spiriconfig_docker.settings*), 68
default (*spiriconfig_docker.settings.Field attribute*), 66
default (*spiriconfig_docker.widgets.Bound property*), 71
DEFAULT_EXEC_COMMAND (*in module spiriconfig_docker.stacks*), 53
description (*spiriconfig.plugins.Plugin attribute*), 37
dev_override (*spiriconfig_docker.stacks.Stack attribute*), 54
DEV_OVERRIDE_FILENAMES (*in module spiriconfig_docker.stacks*), 53
discard_local_command() (*spiriconfig_appstore.stores.Store method*), 76
discover() (*in module spiriconfig_docker.stacks*), 60
discover() (*in module spiriconfig.plugins*), 38
DISCOVERED_WIDGETS (*in module spiriconfig_docker.settings*), 65
docker_bin (*spiriconfig_docker.config.DockerSettings attribute*), 63
docker_settings() (*in module spiriconfig_docker.config*), 63
DockerSettings (*class in spiriconfig_docker.config*), 61
down() (*spiriconfig_docker.stacks.Stack method*), 54

E

enabled() (*in module spiriconfig.advanced*), 39
env (*spiriconfig_docker.settings.Field attribute*), 66
ENV_FILENAME (*in module spiriconfig_docker.settings*), 65
exec() (*spiriconfig_docker.stacks.Stack method*), 55

F

fetch_command() (*spiriconfig_appstore.stores.Store method*), 75
Field (*class in spiriconfig_docker.settings*), 66
find_app() (*in module spiriconfig_appstore.stores*), 78
find_compose_file() (*in module spiriconfig_docker.stacks*), 60
find_dev_override() (*in module spiriconfig_docker.stacks*), 60
find_sidecar() (*in module spiriconfig_docker.settings*), 69
for_stack() (*in module spiriconfig_docker.settings*), 69
form() (*in module spiriconfig_docker.widgets*), 72
format (*spiriconfig_docker.widgets.Widget attribute*), 71

G

get() (*in module spiriconfig_docker.settings*), 69
get() (*in module spiriconfig_docker.stacks*), 61
get() (*spiriconfig.preferences.PreferenceStore method*), 41
get_store() (*in module spiriconfig_appstore.stores*), 79
git_bin (*spiriconfig_appstore.config.AppStoreSettings attribute*), 85

H

hangup() (*spiriconfig_docker.stacks.Stack method*), 56
has_page (*spiriconfig.plugins.Plugin property*), 38
has_settings() (*in module spiriconfig_docker.settings*), 69
help (*spiriconfig_docker.settings.Field attribute*), 66
host (*spiriconfig.config.Settings attribute*), 51

I

icon (*spiriconfig.plugins.Plugin attribute*), 37
in_merge (*spiriconfig_appstore.stores.Store property*), 77
Install (*class in spiriconfig_appstore.installs*), 81
install_command() (*in module spiriconfig_appstore.installs*), 82
install_command() (*spiriconfig_appstore.stores.App method*), 75
installed() (*in module spiriconfig_appstore.installs*), 82
interactive() (*in module spiriconfig.terminal*), 47
is_dirty() (*spiriconfig_appstore.stores.Store method*), 77

L

label (*spiriconfig_docker.settings.Field attribute*), 66
link (*spiriconfig_appstore.installs.Install attribute*), 81

link_path() (*spiriconfig_appstore.stores.App method*), 74
local_diff() (*spiriconfig_appstore.stores.App method*), 74
log_command() (*spiriconfig_appstore.stores.App method*), 74
log_file (*spiriconfig.config.Settings attribute*), 51
log_level (*spiriconfig.config.Settings attribute*), 51
logs() (*spiriconfig_docker.stacks.Stack method*), 55

M

mark() (*in module spiriconfig.advanced*), 40
MARKER_PREFIX (*in module spiriconfig_docker.widgets*), 70
mem_bytes (*spiriconfig_docker.stacks.Usage attribute*), 59
merge_command() (*spiriconfig_appstore.stores.Store method*), 76
model_config (*spiriconfig_appstore.config.AppStoreSettings attribute*), 84
model_config (*spiriconfig_docker.config.DockerSettings attribute*), 63
model_config (*spiriconfig.config.Settings attribute*), 50
module
    spiriconfig_appstore.config, 83
    spiriconfig_appstore.installs, 80
    spiriconfig_appstore.stores, 73
    spiriconfig_docker.config, 61
    spiriconfig_docker.settings, 63
    spiriconfig_docker.stacks, 53
    spiriconfig_docker.widgets, 70
    spiriconfig.advanced, 38
    spiriconfig.commands, 42
    spiriconfig.config, 48
    spiriconfig.plugins, 36
    spiriconfig.preferences, 40
    spiriconfig.terminal, 46

N

name (*spiriconfig_appstore.installs.Install attribute*), 81
name (*spiriconfig_appstore.stores.App attribute*), 73
name (*spiriconfig_docker.stacks.Stack attribute*), 54
name (*spiriconfig.plugins.Plugin attribute*), 37
NEW_COMPOSE_TEMPLATE (*in module spiriconfig_docker.stacks*), 54

O

on_startup() (*spiriconfig.plugins.Plugin method*), 37
only() (*in module spiriconfig.advanced*), 40
options (*spiriconfig_docker.settings.Field attribute*), 66
output() (*spiriconfig.commands.PtySession method*), 44

P

page() (*spiriconfig.plugins.Plugin method*), 37
parse (*spiriconfig_docker.widgets.Widget attribute*), 71
path (*spiriconfig_appstore.stores.App attribute*), 73
path (*spiriconfig_appstore.stores.Store attribute*), 75
path (*spiriconfig_docker.stacks.Stack attribute*), 54
pattern (*spiriconfig_docker.settings.Field attribute*), 66
Plugin (*class in spiriconfig.plugins*), 37
plugin_discovery_interval (*spiriconfig.config.Settings attribute*), 51
port (*spiriconfig.config.Settings attribute*), 51
PREFERENCE_KEY (*in module spiriconfig.advanced*), 39
preferences() (*in module spiriconfig.preferences*), 42
PreferenceStore (*class in spiriconfig.preferences*), 41
preview() (*spiriconfig_docker.settings.StackSettings method*), 67
ps() (*spiriconfig_docker.stacks.Stack method*), 55
PtySession (*class in spiriconfig.commands*), 43
pull() (*spiriconfig_docker.stacks.Stack method*), 55
pump() (*in module spiriconfig.terminal*), 47

R

read() (*spiriconfig_docker.settings.StackSettings method*), 67
read() (*spiriconfig_docker.stacks.Stack method*), 59
REGISTRY (*in module spiriconfig_docker.widgets*), 70
remote_url() (*in module spiriconfig_appstore.stores*), 79
remove_command() (*spiriconfig_appstore.stores.Store method*), 76
render() (*in module spiriconfig_docker.widgets*), 72

reset() (*in module spiriconfig.preferences*), 42
reset() (*spiriconfig_docker.widgets.Bound method*), 71
RESET_PREFIX (*in module spiriconfig_docker.widgets*), 70
resize() (*spiriconfig.commands.PtySession method*), 45
resolve_plan() (*spiriconfig_appstore.stores.Store method*), 77
restart() (*spiriconfig_docker.stacks.Stack method*), 55
Result (*class in spiriconfig.commands*), 42
run() (*in module spiriconfig.commands*), 43
running (*spiriconfig.commands.PtySession property*), 44
running_services() (*spiriconfig_docker.stacks.Stack method*), 58

S

save() (*spiriconfig_docker.settings.StackSettings method*), 68
session_cookie (*spiriconfig.config.Settings attribute*), 52
set() (*spiriconfig.preferences.PreferenceStore method*), 41
set_enabled() (*in module spiriconfig.advanced*), 39
Settings (*class in spiriconfig.config*), 48
settings() (*in module spiriconfig.config*), 53
SETTINGS_KEY (*in module spiriconfig_docker.settings*), 65
SettingsError, 67
SIDECAR_FILENAMES (*in module spiriconfig_docker.settings*), 65
slug_for() (*in module spiriconfig_appstore.stores*), 79
spiriconfig_appstore.config
    module, 83
spiriconfig_appstore.installs
    module, 80
spiriconfig_appstore.stores
    module, 73
spiriconfig_docker.config
    module, 61
spiriconfig_docker.settings
    module, 63
spiriconfig_docker.stacks
    module, 53
spiriconfig_docker.widgets
    module, 70
spiriconfig.advanced
    module, 38
spiriconfig.commands
    module, 42
spiriconfig.config
    module, 48
spiriconfig.plugins
    module, 36
spiriconfig.preferences
    module, 40
spiriconfig.terminal
    module, 46
Stack (*class in spiriconfig_docker.stacks*), 54
StackError, 59
StackSettings (*class in spiriconfig_docker.settings*), 67
start() (*spiriconfig.commands.PtySession method*), 44
state() (*in module spiriconfig.advanced*), 39
stats() (*spiriconfig_docker.stacks.Stack method*), 58
status() (*spiriconfig_docker.stacks.Stack method*), 57
storage_secret (*spiriconfig.config.Settings attribute*), 52
Store (*class in spiriconfig_appstore.stores*), 75
store_dir (*spiriconfig_appstore.config.AppStoreSettings attribute*), 85
store_for_url() (*in module spiriconfig_appstore.stores*), 79
StoreError, 78
stores (*spiriconfig_appstore.config.AppStoreSettings attribute*), 85
stores() (*in module spiriconfig_appstore.stores*), 80
stream() (*in module spiriconfig.commands*), 46
stream_pty() (*in module spiriconfig.commands*), 45

T

target (*spiriconfig_appstore.installs.Install attribute*), 81
terminal() (*in module spiriconfig.terminal*), 48
TERMINAL_ROWS (*in module spiriconfig.terminal*), 47
title (*spiriconfig_docker.settings.Field property*), 67
title (*spiriconfig.plugins.Plugin attribute*), 37
tls (*spiriconfig.config.Settings attribute*), 52

tls_cert (*spiriconfig.config.Settings attribute*), 52
tls_key (*spiriconfig.config.Settings attribute*), 52
toggle() (*in module spiriconfig.advanced*), 40
TRUTHY (*in module spiriconfig_docker.widgets*), 70

U

uninstall() (*in module spiriconfig_appstore.installs*), 82
uninstall_command() (*spiriconfig_appstore.installs.Install method*), 81
unresolved() (*spiriconfig_appstore.stores.Store method*), 77
up() (*spiriconfig_docker.stacks.Stack method*), 54
update_plan() (*in module spiriconfig_appstore.stores*), 80
upstream_diff() (*spiriconfig_appstore.stores.App method*), 74
Usage (*class in spiriconfig_docker.stacks*), 59
usage() (*spiriconfig_docker.stacks.Stack method*), 58
use() (*in module spiriconfig.preferences*), 42

V

validate() (*spiriconfig_docker.stacks.Stack method*), 57
value (*spiriconfig_docker.widgets.Bound property*), 71
values() (*in module spiriconfig_docker.widgets*), 72
values() (*spiriconfig_docker.settings.StackSettings method*), 67
version() (*spiriconfig_appstore.stores.App method*), 73
VERSION_KEY (*in module spiriconfig_appstore.stores*), 78

W

wait() (*spiriconfig.commands.PtySession method*), 45
Widget (*class in spiriconfig_docker.widgets*), 71
WIDGETS (*in module spiriconfig_docker.settings*), 65
write() (*spiriconfig_docker.settings.StackSettings method*), 68
write() (*spiriconfig_docker.stacks.Stack method*), 59
write() (*spiriconfig.commands.PtySession method*), 44